



ndays are also **0days**: Can hackers launch 0day RCE attack on popular software only with chromium ndays?

Bohan Liu, Zheng Wang, Guancheng Li



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB



> whoami



Bohan Liu

- @P4nda20371774
- Security Researcher at Tencent Security Xuanwu Lab
- Mainly Engaged in Browser Security
- Google Chrome Bug Hunter



Zheng Wang

- @xmzyshypnc
- Security Researcher at Tencent Security Xuanwu Lab
- Mainly Engaged in Browser Security and Kernel Security
- Found Several security bugs in Apple Safari, Linux kernel and VirtualBox



Guancheng Li

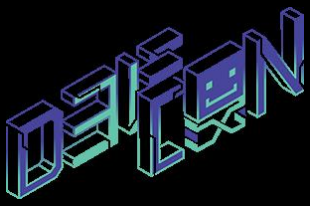
- @atuml1
- Security Researcher at Tencent Security Xuanwu Lab
- co-founder and former captain of r3kapiG
- Previous focus on system/software security, currently focus on LLM x Security



Agenda

- Introduction
- RCE-Window-period in Chromium-based software
- Attack Surface and Vulnerability Discovery
- Achieve RCE Without Sandbox Escape Bug
- Security Recommendations
- Conclusion & Takeaways





0. Introduction

Background

- I researched Chromium bugs for a while and wrote **some Nday exploits**.
- I know **little** web security.
- I can do **a little** reversing.

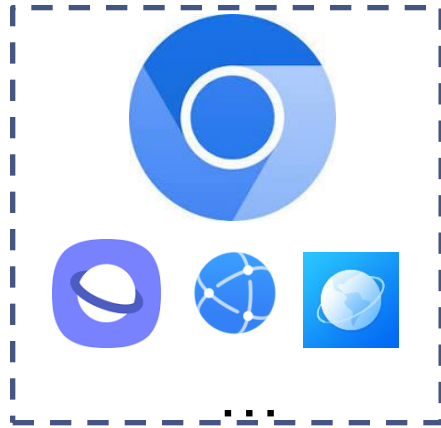


Can I find **0day RCE bugs** in popular software rather than Chrome?

How can I make it **faster and easier**?



How does chromium appears in software?



Chromium-based development



Electron framework



CEF



QtWebEngine

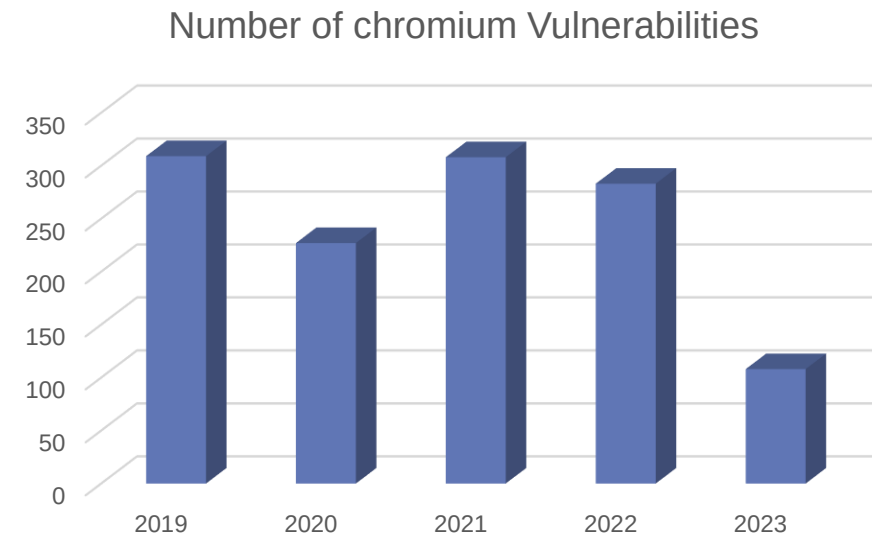
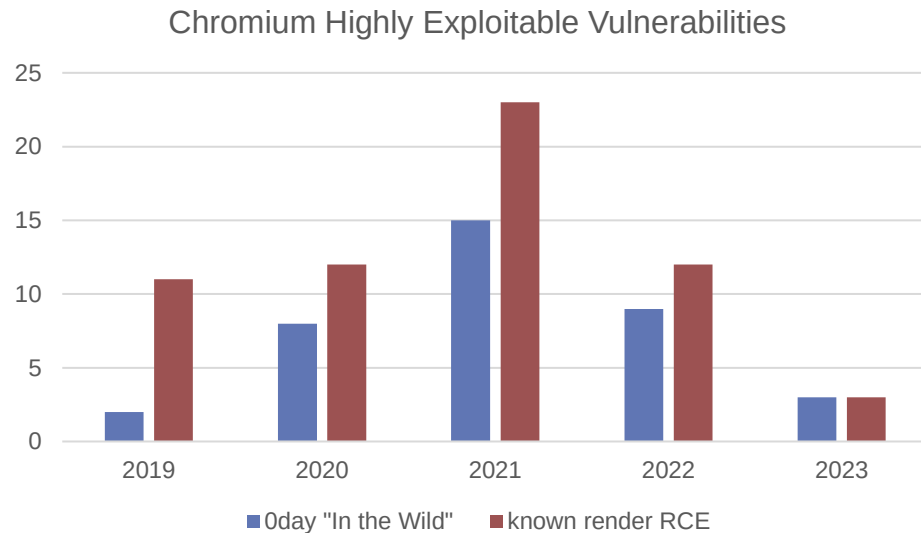


Other Supply Chain



The hidden dangers of chromium-based software

Chromium Highly Exploitable Vulnerabilities



[1] https://www.cvedetails.com/product/15031/Google-Chrome.html?vendor_id=1224
[2] <https://googleprojectzero.blogspot.com/p/0day.html>



The hidden dangers of chromium-based software

- Google's vulnerability disclosure strategy

Investigating and reporting bugs

All bugs should be reported via [this form](#), which will apply the correct labels and view restrictions. Note that your submission is over HTTPS and does not require additional encryption. Bugs that are found in Google's server-side services should be reported under the [Google Vulnerability Rewards Program](#) instead.

When investigating a vulnerability, please, only ever target your own computers. Never attempt to access anyone else's data and do not engage in any activity that would be disruptive or damaging to your fellow users or to Google.

Note that we are only able to respond to technical vulnerability reports. Chromium embedders and companies with whom Google has a pre-existing business relationship may not be eligible for rewards. Non-security bugs and queries about problems with your account should be instead directed to [Google Help Centers](#).

Reports are made public 14 weeks after being marked as fixed, other than in exceptional cases. This is in keeping Chromium's open source philosophy, and provides a valuable resource for conducting vulnerability research. Note that attachments are considered part of the report.

[3] <https://bughunters.google.com/about/rules/5745167867576320/chrome-vulnerability-reward-program-rules>



The hidden dangers of chromium-based software

```
chromium / v8 / v8 / 02f84c745fc0cae5927a66dc4a3e81334e8f60a6

commit 02f84c745fc0cae5927a66dc4a3e81334e8f60a6 [log] [tgz]
author Georg Neis <neis@chromium.org> Mon Apr 12 07:42:03 2021
committer Commit Bot <commit-bot@chromium.org> Mon Apr 12 10:05:42 2021
tree 3022494d5e57e4f79cacb8a285e53be835d87207
parent 1e4b1c521a491c7487028b7f2aec550c1b36606b [diff]

[compiler][x64] Fix bug in InstructionSelector::ChangeInt32ToInt64

Bug: chromium:1196683
Change-Id: Ib4ea738b47b54edc81450583be4c80a41698c3d1
Reviewed-on: https://chromium-review.googlesource.com/c/v8/v8/+2820971
Commit-Queue: Georg Neis <neis@chromium.org>
Reviewed-by: Nico Hartmann <nicohartmann@chromium.org>
Cr-Commit-Position: refs/heads/master@{#73903}

src/compiler/backend/x64/instruction-selector-x64.cc [diff]
test/mjsunit/compiler/regress-1196683.js [Added - diff]
```

```
chromium / v8 / v8 / fd29e246f65a7cee130e72cd10f618f3b82af232

commit fd29e246f65a7cee130e72cd10f618f3b82af232 [log] [tgz]
author Georg Neis <neis@chromium.org> Mon Apr 12 10:53:24 2021
committer Commit Bot <commit-bot@chromium.org> Mon Apr 12 15:02:33 2021
tree 7c5af2534438e3907f82e7898e551462179fa61d
parent f87baad0f8b3f7fda43a01b9af3dd940dd09a9a3 [diff]

[compiler] Fix bug in RepresentationChanger::GetWord32RepresentationFor

We have to respect the TypeCheckKind.

Bug: chromium:1195777
Change-Id: Ifeed719fef79b7c61d99c29ba869dd7985c413
Reviewed-on: https://chromium-review.googlesource.com/c/v8/v8/+2817791
Commit-Queue: Georg Neis <neis@chromium.org>
Reviewed-by: Nico Hartmann <nicohartmann@chromium.org>
Cr-Commit-Position: refs/heads/master@{#73909}

src/compiler/representation-change.cc [diff]
test/mjsunit/compiler/regress-1195777.js [Added - diff]
```

 **Rajvardhan Agarwal**
@r4j0x00

Just here to drop a chrome Oday. Yes you read that right.
github.com/r4j0x00/exploi...

Catalin Cimpanu
April 20th, 2021

News Technology



Recent Chromium bug used to attack Chinese WeChat users

A Chrome exploit published online last week has been weaponized and abused to attack WeChat users in China, a local security firm reported on Friday.

The attacks consisted of malicious links sent to WeChat users.

If users clicked the link, they'd trigger a piece of JavaScript code that would load and execute **shellcode** on their underlying operating systems.

Qingteng Cloud Security, which detected the attacks via one of its security products, said in a Weibo post that only users of the **WeChat Windows app** were attacked.

 **frust**
@frust93717815

another chrome Oday
github.com/avboy1337/1195...
Just here to drop a chrome Oday. Yes you read that right.
[翻译推文](#)

下午7:56 · 2021年4月14日

Hackers have lots of priori knowledge for chromium-based software attacks.



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB



1. “RCE Window Period” in Chromium-based software

RCE Window Period in software

Definition : Patch gap between the fix of an exploitable bug on chromium and on software :

$$T_{RWPeriod} = t_{fix} - t_{exploitable} \quad (1)$$

$$T_{UpdateGap} = t_{software_update} - t_{chromium_update} \quad (2)$$



...



RCE Window Period in software

Method :

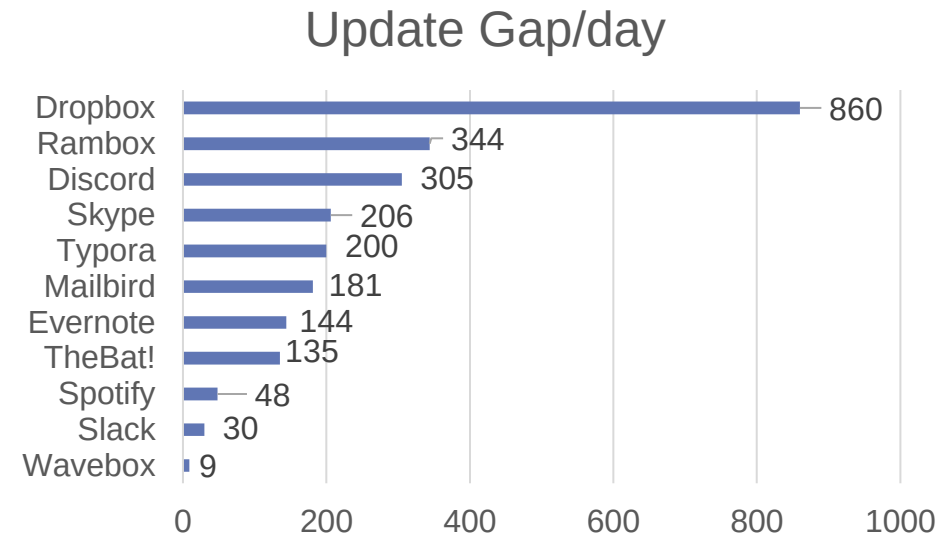
- Download the installation package for older versions
- Identify the type and version of the Chromium component in the software.
- Calculate the update cycle of software, the bug list and its RCE window period



RCE Window Period in software

Result :

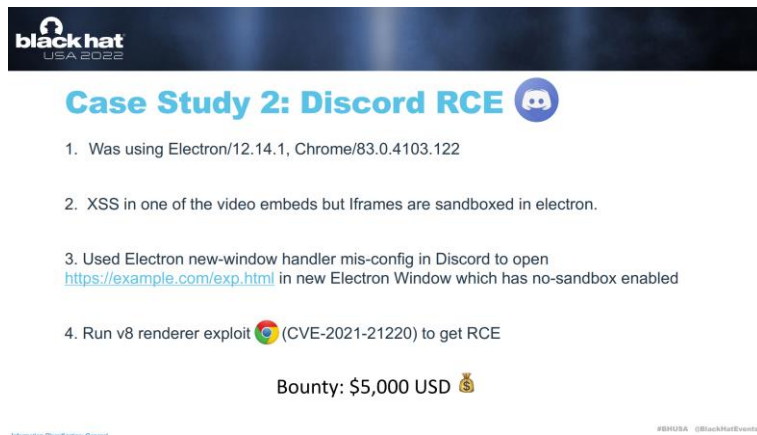
The patch period varies from 9 to 800 days between software



RCE Window Period in software

Result :

For Discord, the update gap is as long as 350 days and there's lots of nday-exp for attackers to use.



black hat
USA 2022

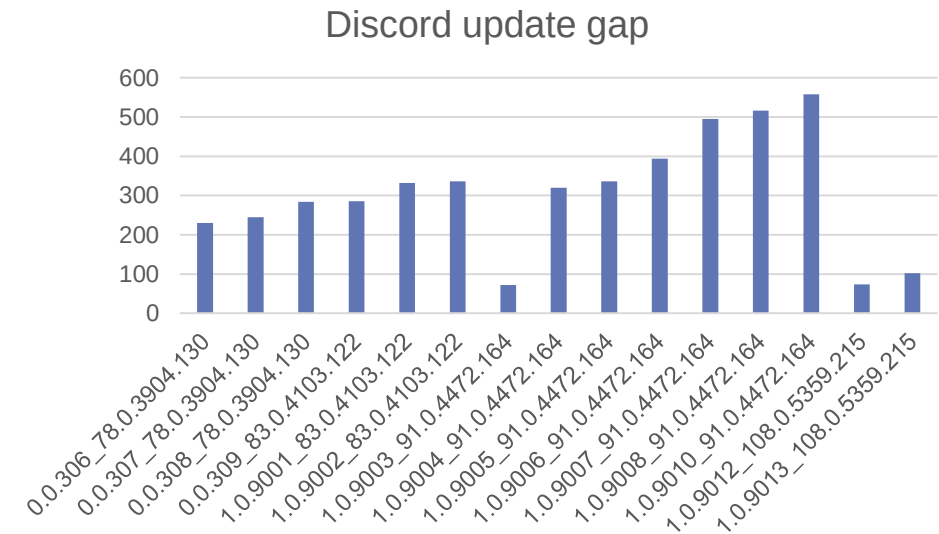
Case Study 2: Discord RCE

1. Was using Electron/12.14.1, Chrome/83.0.4103.122
2. XSS in one of the video embeds but Iframes are sandboxed in electron.
3. Used Electron new-window handler mis-config in Discord to open <https://example.com/exp.html> in new Electron Window which has no-sandbox enabled
4. Run v8 renderer exploit (CVE-2021-21220) to get RCE

Bounty: \$5,000 USD

Information Classification: General

#BHUSA @BlackHatEvents

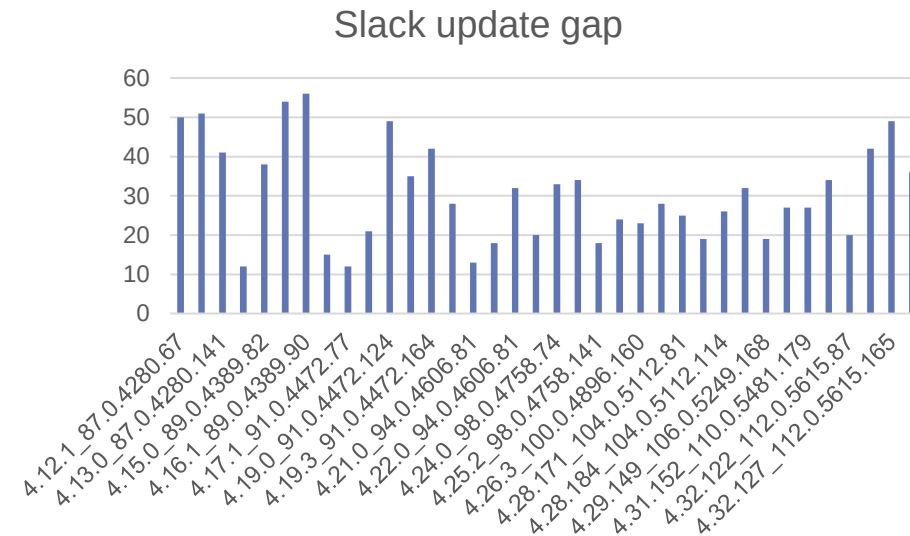
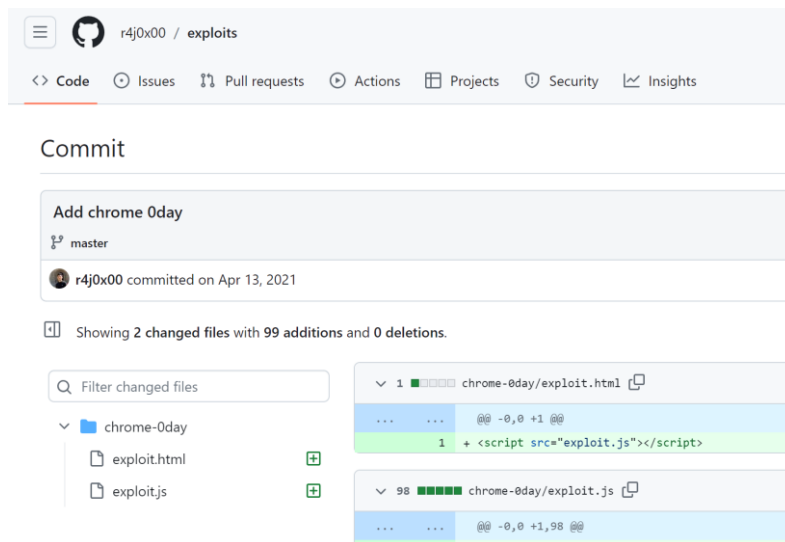


[4] <https://blog.electrovolt.io/posts/discord-rce/>

RCE Window Period in software

Result :

For Slack, the update gap is 30 days but it still at risk with unexpected 0-day disclosure



[5] <https://github.com/r4j0x00/exploits/commit/7ba55e5ab034d05877498e83f144e187d3ddb160>



RCE Window Period in software

Conclusion :

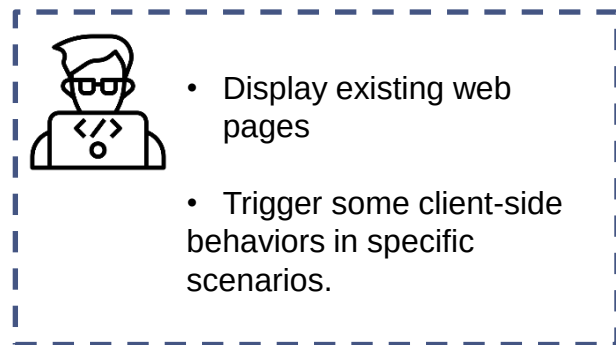
- The RCE Window Period exists universally in modern software
- The electron-based application updates more often than others
- Although many software tries to lower the patch period since a specific time, there's still an opportunity for exploitation



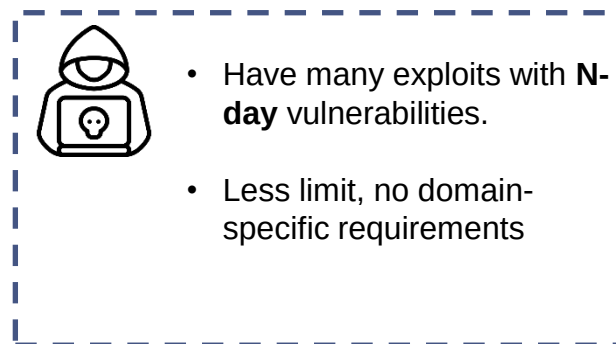


2. Attack Surface and Vulnerability Discovery

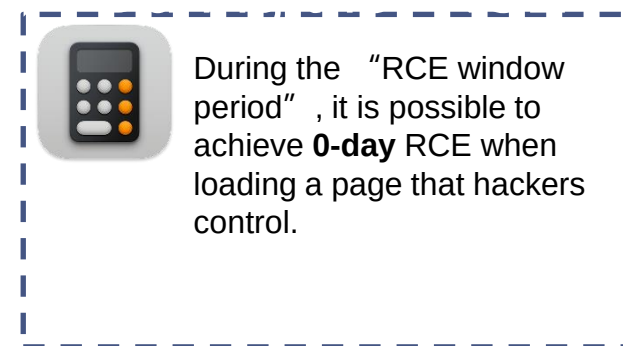
Attack Vector



Premise



Advantage



Result

What are the scenarios that developers tend to overlook?



Attack Vector



What are the scenarios that developers tend to overlook?

- What is the function of browser component?
- What code runs in the browser component?



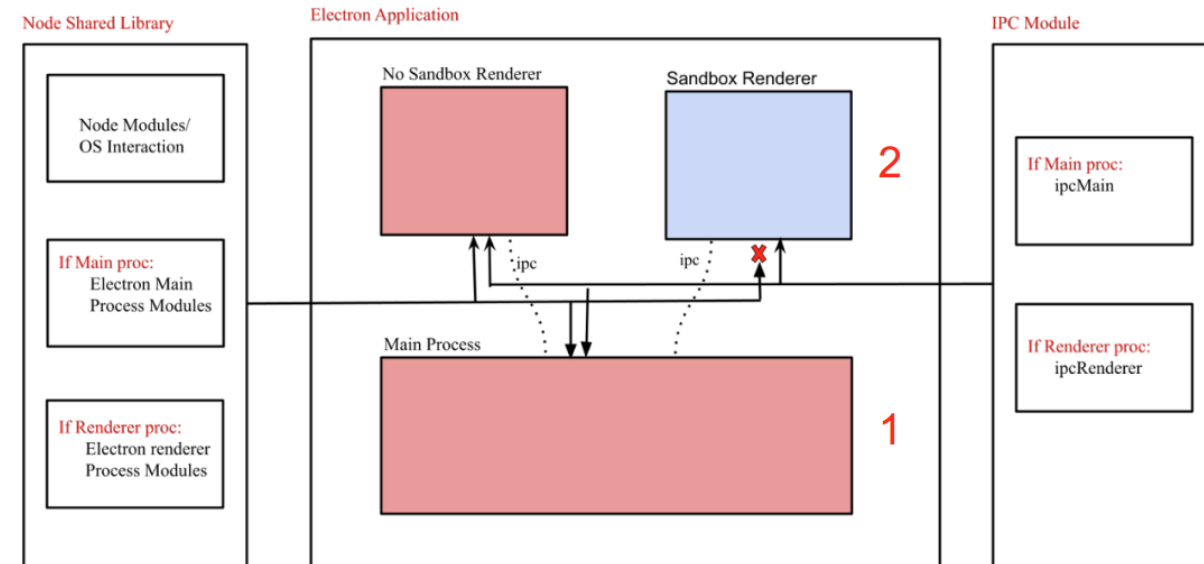
Let's take a look at how to develop it!



The development mode of the browser framework

Electron

- All codes are developed by Nodejs
- Divided into Main Process and Render process, communicate through IPC
- All code is stored in the resource folder
- The Electron framework is packaged in an executable file
- Less modification to Electron Framework code



How to locate and debug

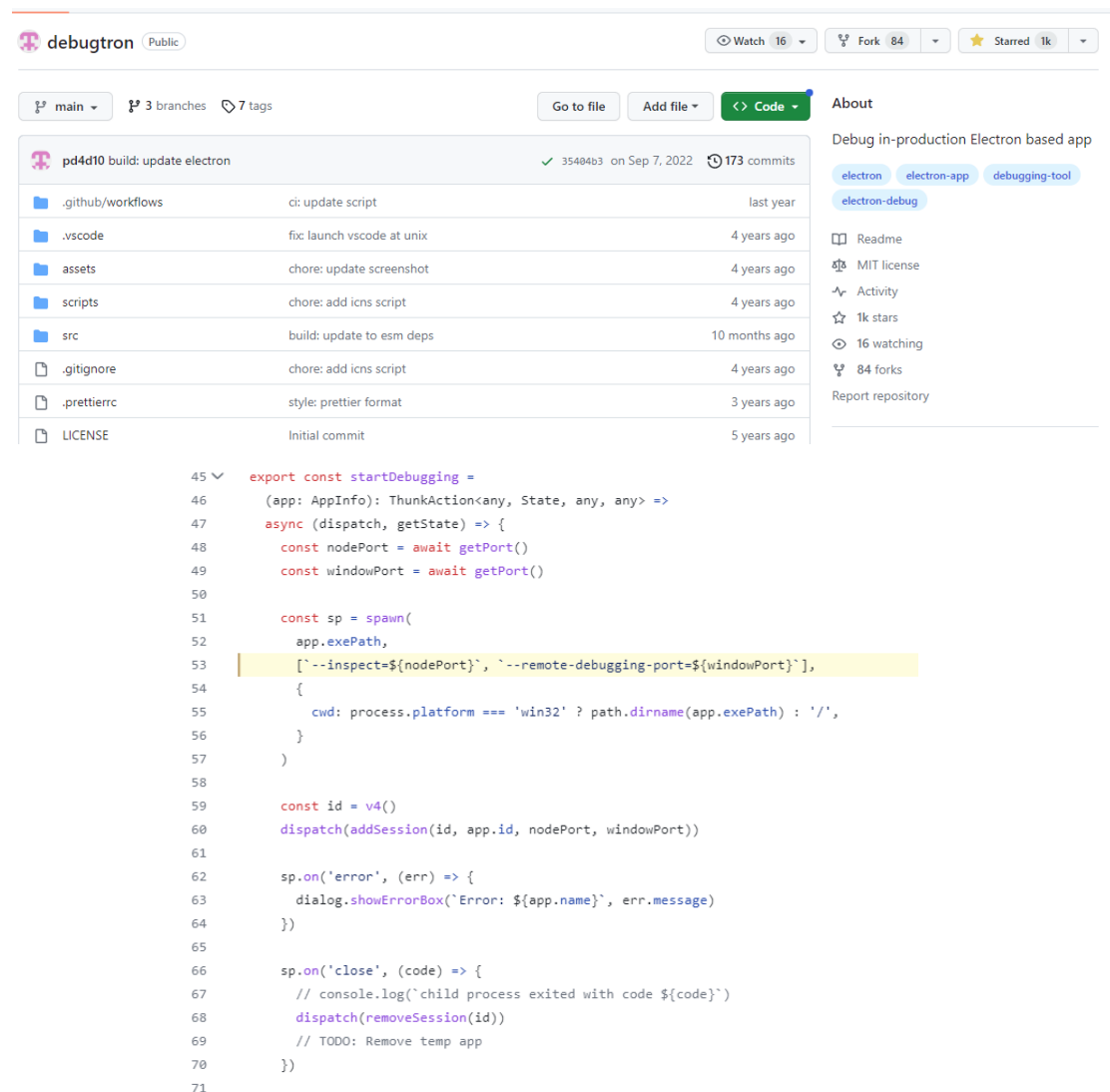
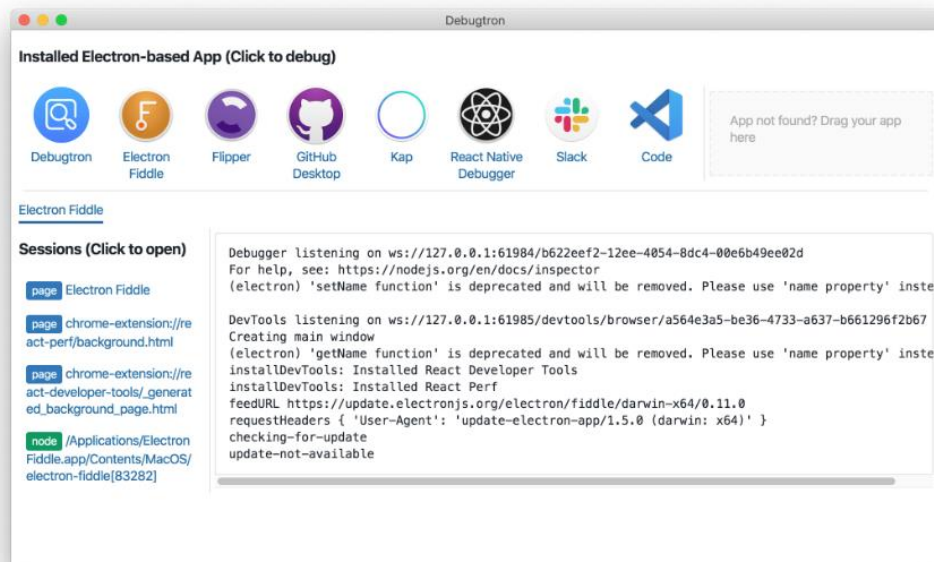
Electron

Locate:

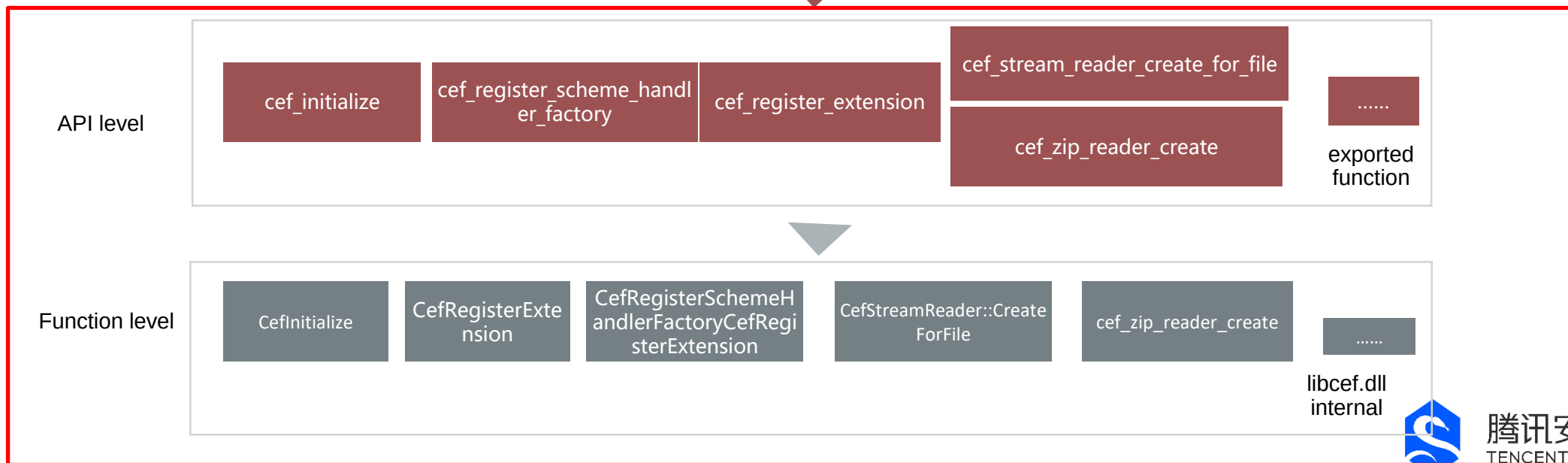
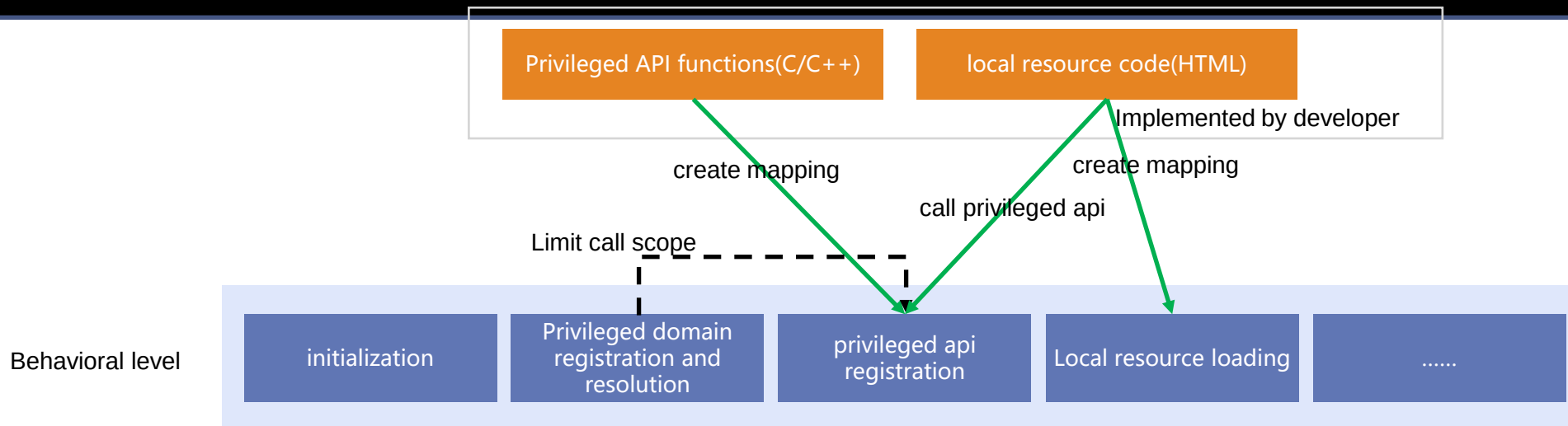
asar extract app.asar xxx

Debug:

Debugtron is an app to debug in-production Electron based app. It is also built with Electron.



CEF Architecture



How to locate and debug

CEF

- Privileged API is implemented by C/C++, local resource code is implemented by HTML.
- Exists in the form of a dynamic link library.
- The location of the local resource file is uncertain, and the file may be encrypted.
- Flexible usage of CEF and possible modifications to the library.



Any common debugging method?



Hook the API in libcef,
get and modify what we want



How to locate and debug

Locate

- Development mode:
cef_stream_reader_create_for_file + cef_zip_reader_create + zip_reader_open_file

```
NO_SANITIZE("cfi-icall")
struct _cef_stream_reader_t* cef_stream_reader_create_for_file(
    const cef_string_t* fileName) {
    return g_libcef_pointers.cef_stream_reader_create_for_file(fileName);
}
```

```
NO_SANITIZE("cfi-icall")
struct _cef_zip_reader_t* cef_zip_reader_create(
    struct _cef_stream_reader_t* stream) {
    return g_libcef_pointers.cef_zip_reader_create(stream);
}
```

```
int CEF_CALLBACK zip_reader_open_file(struct _cef_zip_reader_t* self,
    const cef_string_t* password) {
    shutdown_checker::AssertNotShutdown();

    // AUTO-GENERATED CONTENT - DELETE THIS COMMENT BEFORE MODIFYING

    DCHECK(self);
    if (!self) {
        return 0;
    }
    // Unverified params: password

    // Execute
    bool _retval = CefZipReaderCppToC::Get(self)->OpenFile(CefString(password));

    // Return type: bool
    return _retval;
}
```

```
C:\Windows\System32\cmd.exe - frida | detect_resource_x64.js --no-pause
finish open
[+] detect_resource : eb_content.dat
call cef_zip_reader_create finish
password : b6ab3c574
finish open
A
```


How to locate and debug

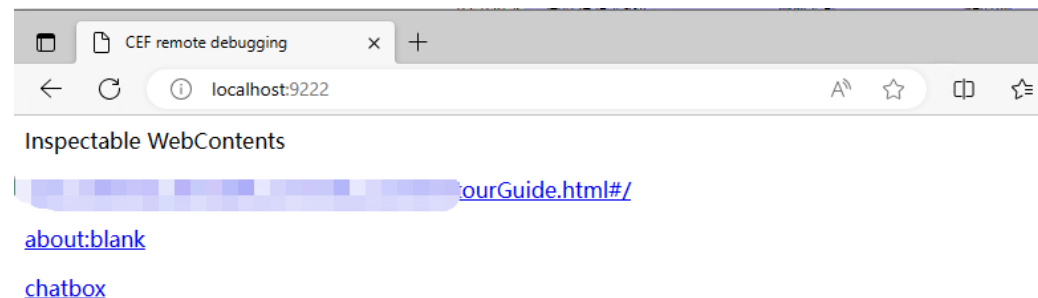
Debug

- Development mode:

cef_initialize with static configuration parameters

```
NO_SANITIZE("cfi-icall")
int cef_initialize(const cef_main_args_t* args,
                  const struct _cef_settings_t* settings,
                  struct _cef_app_t* application,
                  void* windows_sandbox_info) {
    return g_libcef_pointers.cef_initialize(args, settings, application,
                                           windows_sandbox_info);
}
```

```
160 typedef struct _cef_settings_t {
161     ///
162     /// Size of this structure.
163     ///
164     size_t size;
165
166     ///
167     /// Set to true (1) to disable the sandbox for sub-processes. See
168     /// cef_sandbox_win.h for requirements to enable the sandbox on Windows. Also
169     /// configurable using the "no-sandbox" command-line switch.
170     ///
171     int no_sandbox;
172     ...
173
374     ///
375     /// Set to a value between 1024 and 65535 to enable remote debugging on the
376     /// specified port. Also configurable using the "remote-debugging-port"
377     /// command-line switch. Remote debugging can be accessed by loading the
378     /// chrome://inspect page in Google Chrome. Port numbers 9222 and 9229 are
379     /// discoverable by default. Other port numbers may need to be configured via
380     /// "Discover network targets" on the Devices tab.
381     ///
382     int remote_debugging_port;
383 }
```



We can also build more powerful debug tools based on development mode



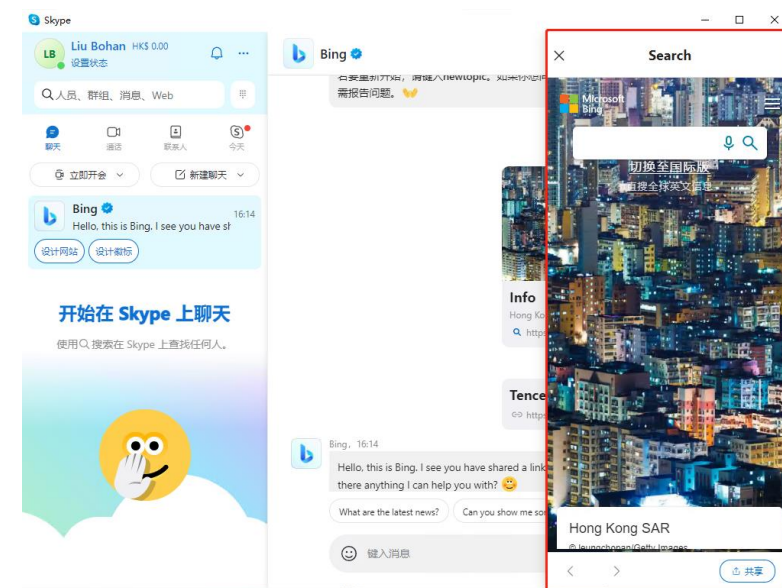
腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Scenario 1: Whitelist of allowed loads

Development logic:

User clicks on URL

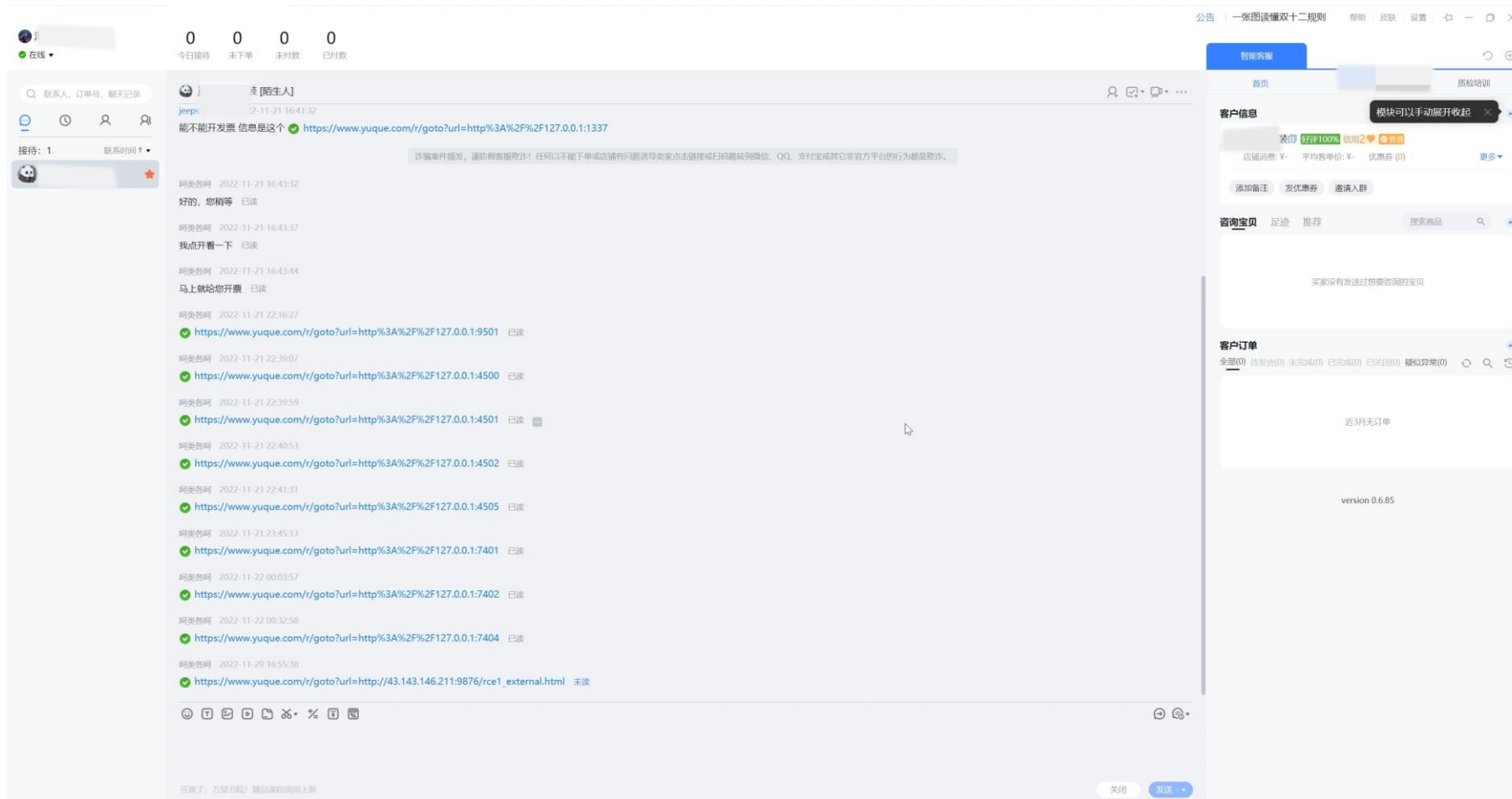
- Check if URL's domain is in whitelist?
- if yes, load it in the software's internal browser component
- if not, Call `shellExecute("open",url,...)` to open the URL with default browser



Scenario 1: Whitelist of allowed loads

How to bypass?

- Bypass check via Page redirection (demo)



Scenario 1: Whitelist of allowed loads

How to bypass?

- Incomplete whitelist checking
 - Incorrect regular expression
 - Incomplete Triple Check

```
1 int FilterUrl()
2 {
3     JMUrlUtils::g_strRegJdUrl = QString::fromAscii_helper(
4         "(^[\\S]*(jd|jdpay|jcloud|jdccloud|jdw1|360buyimg|kuaidi100|300hu)\\.com[\\S]*$)|(^[\\S]*("
5         "(3|jd1|wjx)\\.cn|jd\\.hk)[\\S]*$)",
6         114);
7     return sub_100018B6(sub_100278E0);
8 }
```

GP

"(^[\\S]*(jd|jdpay|jcloud|jdccloud|jdw1|360buyimg|kuaidi100|300hu)\\.com[\\S]*\$)|(^[\\S]*((3|jd1|wjx)\\.cn|jd\\.hk)[\\S]*\$)" What strings can this regular expression match?



The given regular expression:

```
`(^[\\S]*(jd|jdpay|jcloud|jdccloud|jdw1|360buyimg|kuaidi100|300hu)\\.com[\\S]*$)|(^[\\S]*((3|jd1|wjx)\\.cn|jd\\.hk)[\\S]*$)`
```

This regular expression can match strings that fall into the following patterns:

1. Any string that contains one of the following substrings, followed by the ".com" domain extension:
 - jd
 - jdpay
 - jcloud
 - jdccloud
 - jdw1
 - 360buyimg
 - kuaidi100
 - 300hu
2. Any string that contains one of the following substrings, followed by either ".cn" or ".hk":
 - 3
 - jd1
 - wjx

For example: "examplejd.com", "abcjdpay.com", "xyz360buyimg.com"

For example: "example3.cn", "abcjd.hk", "xyzjd1.cn"

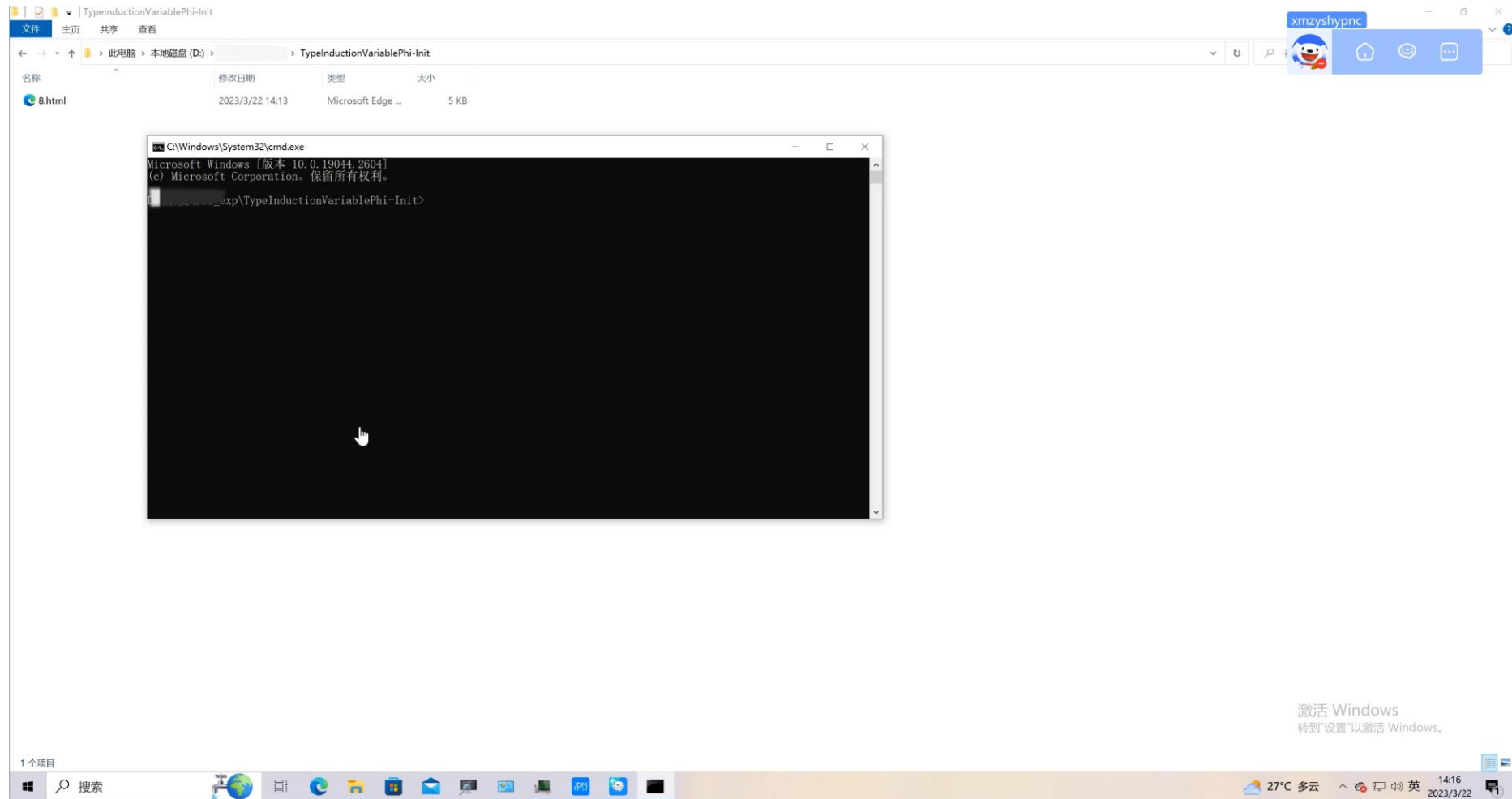
Please note that the regular expression matches patterns in the given strings but does not guarantee that the matched strings are valid URLs or exist in reality.



Scenario 1: Whitelist of allowed loads

How to bypass?

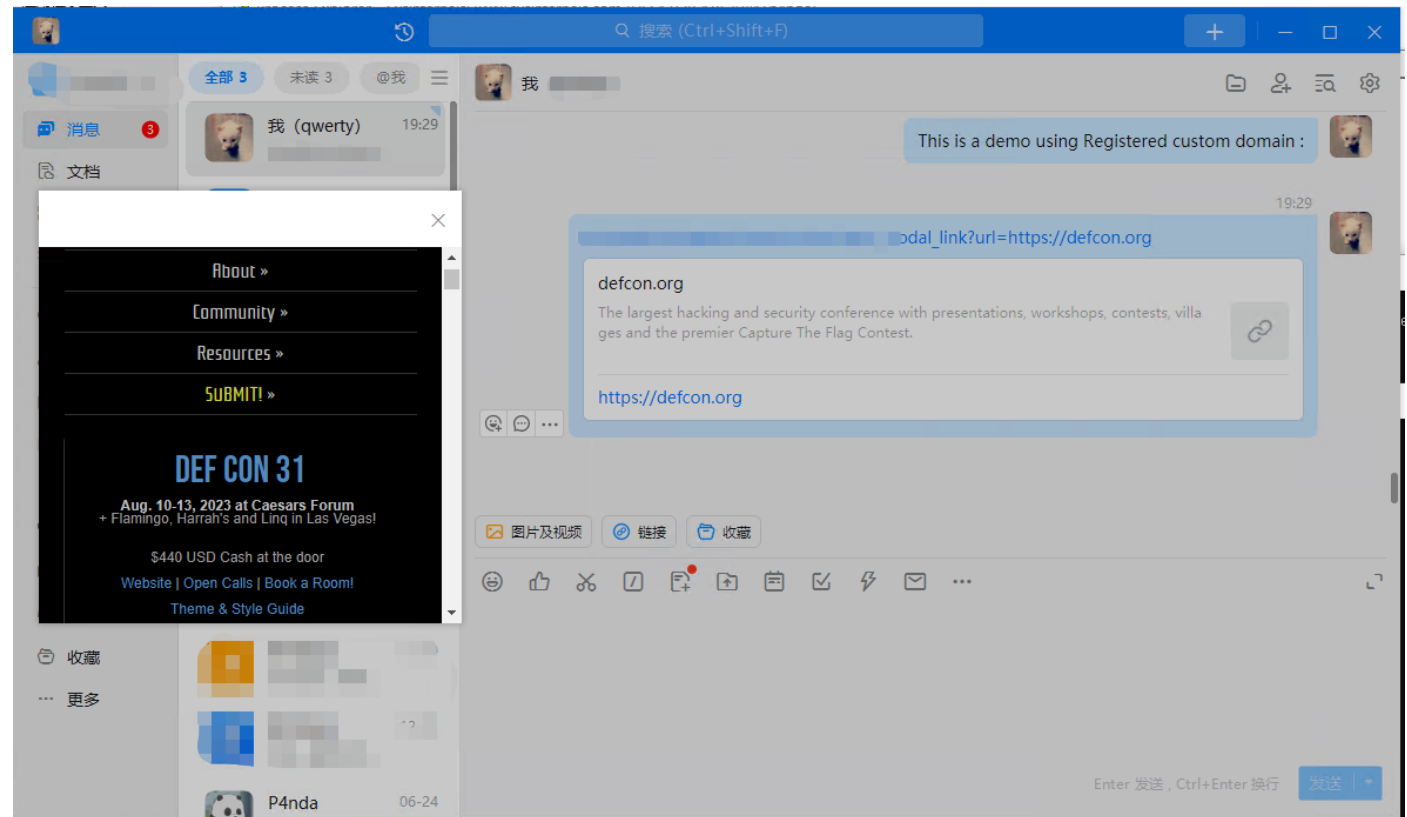
- Incomplete whitelist checking (demo)



Scenario 1: Whitelist of allowed loads

How to bypass?

- Special URL handler
 - Scheme
 - Registered custom domain



Scenario 1: Whitelist of allowed loads

How to bypass?

- Special URL handler (demo)

The screenshot displays a Windows desktop environment. On the left, the Task Manager window is open, showing a list of running processes. The 'Processes' tab is selected, and the list includes various system and user applications. The 'CPU' column is highlighted in blue. On the right, a WeChat chat window is open, showing a conversation with a contact named '我 (qwerty)'. The chat history is visible, and a large, stylized character is displayed in the chat area. The bottom of the screen shows the Windows taskbar with several icons, including the Start button, search bar, and various application icons. The system tray at the bottom right shows the date and time as 06:24 on 06-24.

Process	CPU	Private B...	Working Set	PID	Description	Company Name	Integrity
svchost.exe	13,908 K	22,860 K	17204	Windows 服务主进程	Microsoft Corporation	Medium	
svchost.exe	3,628 K	12,428 K	17024	Windows 服务主进程	Microsoft Corporation	Medium	
svchost.exe	4,172 K	15,292 K	17430	Windows 服务主进程	Microsoft Corporation	Medium	
svchost.exe	9,640 K	14,890 K	9881	Local Security Auth...	Microsoft Corporation	Medium	
fontdrvhost.exe	1,540 K	204 K	1160				
csrss.exe	1,728 K	960 K	922				
winlogon.exe	2,400 K	2,476 K	116				
fontdrvhost.exe	1,260 K	208 K	1152				
csrss.exe	14,084 K	16,012 K	1568				
winlogon.exe	19,994 K	14,124 K	1570				
csrss.exe	2,898 K	3,912 K	3908				
winlogon.exe	2,822 K	3,028 K	3956				
fontdrvhost.exe	3,228 K	10,352 K	3904				
csrss.exe	141,590 K	351,836 K	4120				
csrss.exe	173,360 K	468,244 K	2398	Windows 服务主进程	Microsoft Corporation	Medium	
SecurityHealthSystray.exe	1,908 K	3,684 K	8429	Windows Security noti...	Microsoft Corporation	Medium	
csrss.exe	20,504 K	29,180 K	10544	Microsoft OneDrive	Microsoft Corporation	Medium	
Skype.exe	32,492 K	43,800 K	10938	Skype	Skype Technologies ...	Medium	
Skype.exe	13,224 K	16,524 K	11020	Skype	Skype Technologies ...	Medium	
Skype.exe	110,216 K	21,496 K	10322	Skype	Skype Technologies ...	Low	
Skype.exe	14,468 K	21,140 K	10112	Skype	Skype Technologies ...	Medium	
EvernoteClipper.exe	3,458 K	14,516 K	12332	Evernote Clipper	Beijing Yinyang Bi...	Medium	
csrss.exe	2,380 K	2,188 K	2994	Windows 命令处理程序	Microsoft Corporation	Medium	
csrss.exe	4,780 K	4,440 K	4242	Windows 命令处理程序	Microsoft Corporation	Medium	
csrss.exe	4,268 K	10,896 K	9864	Sysinternals Process ...	Sysinternals - www...	Medium	
csrss.exe	40,760 K	34,208 K	8932	Sysinternals Process ...	Sysinternals - www...	Medium	
csrss.exe	2,460 K	1,416 K	10564	Windows 命令处理程序	Microsoft Corporation	Medium	
csrss.exe	7,996 K	3,328 K	12688	Windows 命令处理程序	Microsoft Corporation	Medium	
photosynth.exe	35,668 K	41,668 K	8768	WPA Office	Zhuai Kingsoft Off...	Medium	
photosynth.exe	16,016 K	39,820 K	11648	WPA Office	Zhuai Kingsoft Off...	Medium	
photosynth.exe	18,996 K	54,392 K	6016	WPA Office	Zhuai Kingsoft Off...	Medium	
photosynth.exe	17,116 K	20,240 K	7328	WPA Office	Zhuai Kingsoft Off...	Medium	
photosynth.exe	12,724 K	19,820 K	11276	WPA Office	Zhuai Kingsoft Off...	Medium	
photosynth.exe	15,016 K	14,940 K	12424	WPA Office	Zhuai Kingsoft Off...	Medium	
photosynth.exe	31,168 K	14,992 K	8292	WPA Office	Zhuai Kingsoft Off...	Medium	
photosynth.exe	30,772 K	24,296 K	2432	Sublime Text	Sublime HQ Pty Ltd	Medium	
photosynth.exe	17,428 K	5,104 K	8220				
photosynth.exe	37,000 K	47,004 K	10692	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	63,116 K	9,044 K	8120	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	30,216 K	2,664 K	1940	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	2,320 K	10,240 K	11012	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	20,480 K	2,676 K	11732	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	32,520 K	22,232 K	10688	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	20,640 K	2,760 K	7884	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	40,208 K	5,168 K	14308	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	39,316 K	6,284 K	14036	迅雷影音	深圳市迅雷网络技术...	Medium	
photosynth.exe	5,716 K	29,616 K	8776	Process Monitor	Sysinternals - www...	Medium	
photosynth.exe	121,240 K	105,716 K	1128				
photosynth.exe	2,400 K	4,768 K	11916	Windows 命令处理程序	Microsoft Corporation	Medium	
photosynth.exe	1,164 K	17,748 K	8030	Windows 命令处理程序	Microsoft Corporation	Medium	
photosynth.exe	26,280 K	30,892 K	8788	Node.js	Node.js	Medium	
photosynth.exe	1,960 K	8,516 K	5792				
photosynth.exe	1,960 K	292 K	732				
photosynth.exe	101,528 K	126,180 K	11836	Microsoft Edge	Microsoft Corporation	Medium	
photosynth.exe	2,120 K	2,612 K	12004	Microsoft Edge	Microsoft Corporation	Medium	
photosynth.exe	99,194 K	20,332 K	11658	Microsoft Edge	Microsoft Corporation	Low	
photosynth.exe	19,736 K	25,200 K	11658	Microsoft Edge	Microsoft Corporation	Medium	
photosynth.exe	10,280 K	5,800 K	11658	Microsoft Edge	Microsoft Corporation	Untrusted	
photosynth.exe	107,752 K	12,020 K	12234	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	9,120 K	11,120 K	3972	Microsoft Edge	Microsoft Corporation	Low	
photosynth.exe	79,620 K	13,080 K	8724	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	278,656 K	19,660 K	12804	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	17,176 K	4,080 K	16020	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	106,824 K	35,632 K	12798	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	8,244 K	4,912 K	16120	Microsoft Edge	Microsoft Corporation	Low	
photosynth.exe	205,444 K	156,372 K	16136	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	106,824 K	32,796 K	14040	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	85,708 K	96,148 K	14888	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	22,924 K	31,684 K	4132	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	88,844 K	100,268 K	8724	Microsoft Edge	Microsoft Corporation	AppContainer	
photosynth.exe	51,476 K	91,016 K	9980	Microsoft Edge	Microsoft Corporation	AppContainer	

激活 Windows
转到“设置”以激活 Windows。



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Scenario 2: Page code injection

What's the difference of XSS between Web Security and chromium-based software?

- XSS type
- Vulnerability discovery method
- Input source



Scenario 2: Page code injection

Local Resource XSS

- Decrypt Local resource
- Find file without CSP
- Find controlled input
- Trigger XSS

```
var ErrorPageObj = {
  reloadIfOnline: function() {
    window.addEventListener("online",function(e){errorpageobj.refreshPage();});
  },
  // ...
  getUrlParam: function(name) {
    const reg = new RegExp(`&?${name}=(^&#)*`);
    const r = window.location.href.match(reg);    1
    if (r !== null) return window.decodeURI(r[1]);
    return '';
  },
  setDarkColor: function() {
    var theme = this.getUrlParam("theme");
    // ...
  },
  setErrorInfo: function() {
    var encodeJsonString = this.getUrlParam("errorInfo");
    var jsonString = decodeURIComponent(encodeJsonString);
    var errorInfo = JSON.parse(jsonString);
    document.title = errorInfo["pageTitle"] || "";
    window.document.getElementsByClassName('msg')[0].innerHTML = errorInfo["msg"] || "";    2
  }
}
```



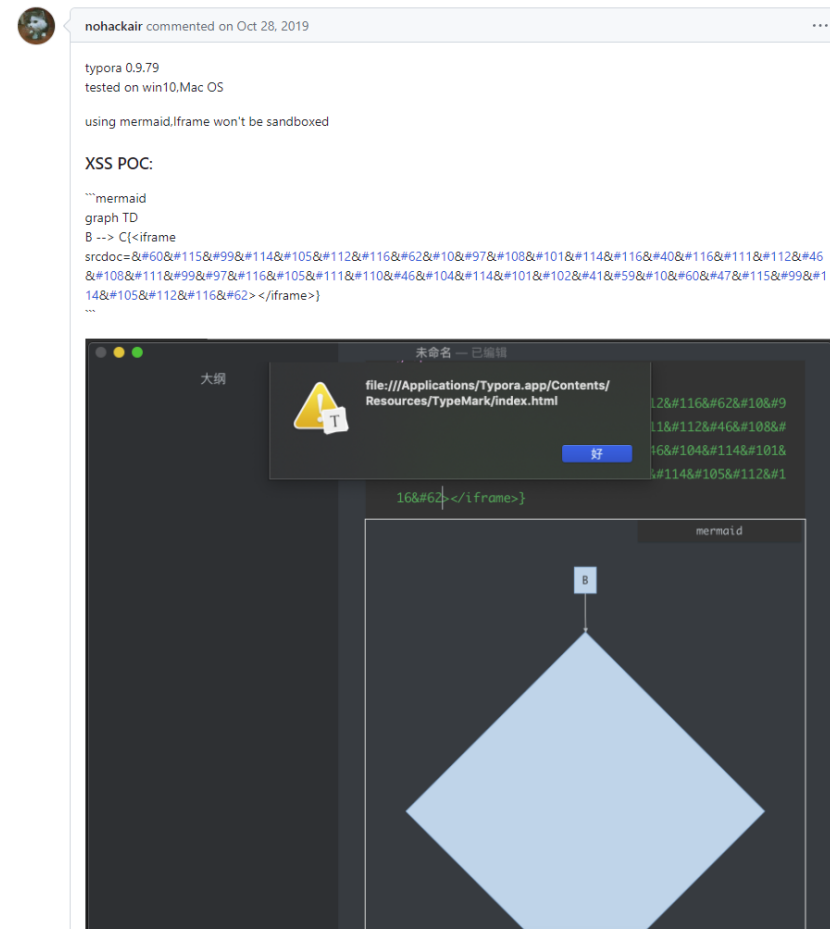
Scenario 2: Page code injection

Third-Party Library XSS (case study)

- Mermaid XSS affecting Typora
- Used to be exploited by privileged API
- Using a Chromium Nday (CVE-2017-5070)

typora(0.9.79) XSS to RCE #2959

 Closed nohackair opened this issue on Oct 28, 2019 · 2 comments



[9] <https://github.com/typora/typora-issues/issues/2959>

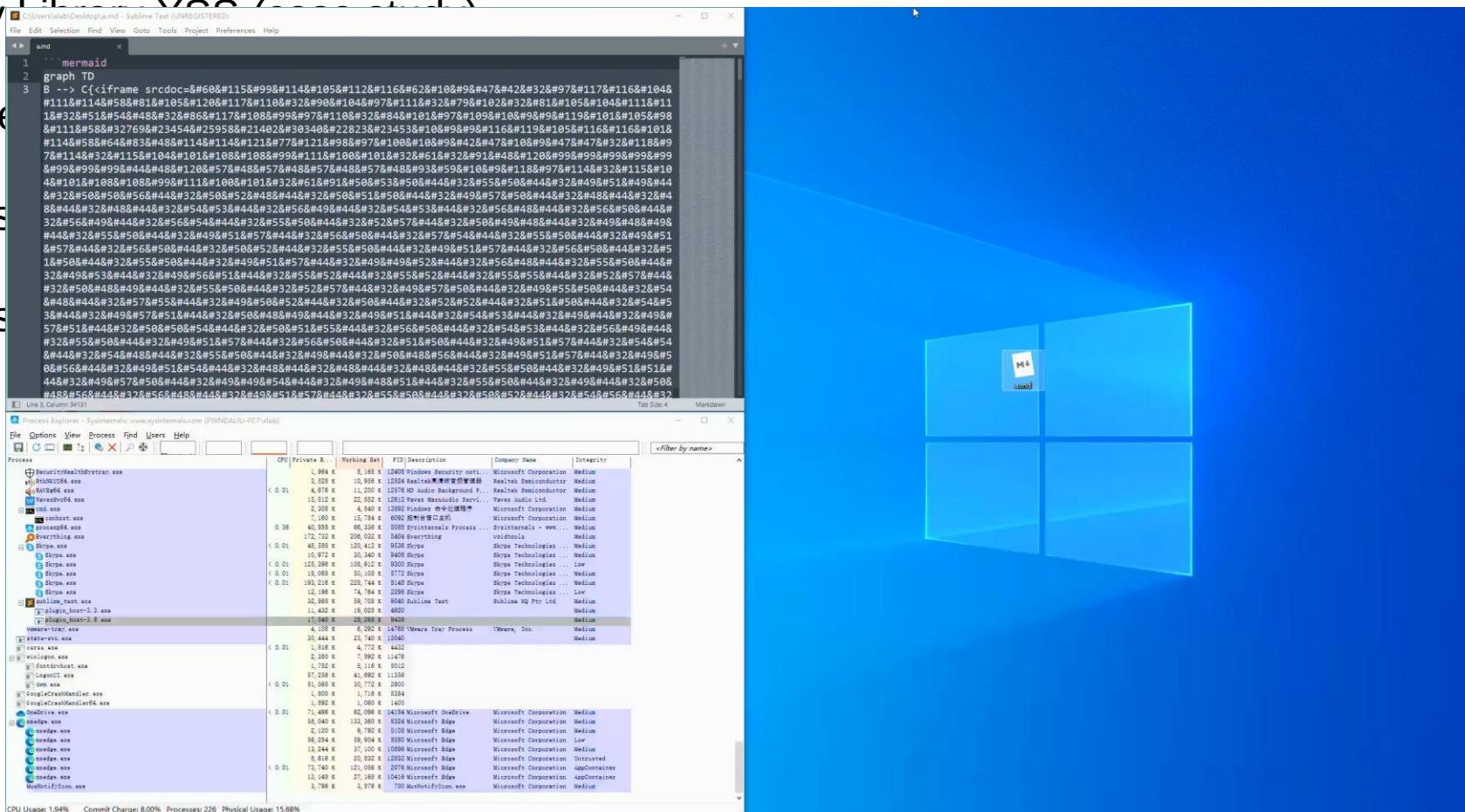
[10] <https://bugs.chromium.org/p/chromium/issues/detail?id=722756>



Scenario 2: Page code injection

Third-Party Libraries (Vulnerable)

- Me
- Us
- Us



[9] <https://github.com/typora/typora-issues/issues/2959>
[10] <https://bugs.chromium.org/p/chromium/issues/detail?id=722756>



Scenario 2: Page code injection

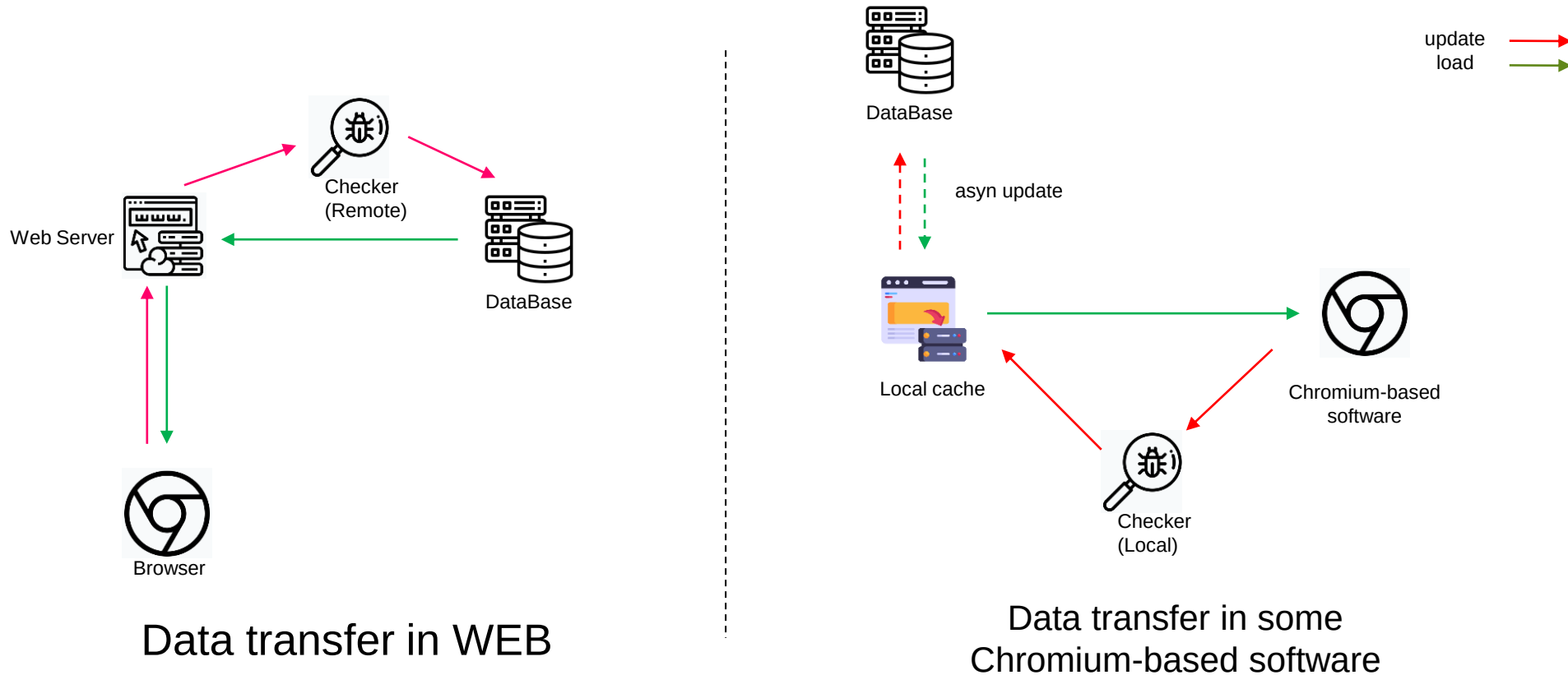
Iframe in chromium-based software

- convenient feature in note/doc software
- No privileged API in iframe
- More dangerous than Web security
- The same logic as loading URL



Scenario 3: Cache data injection

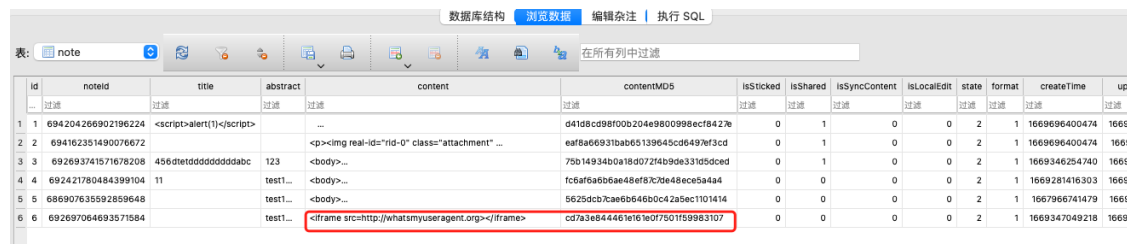
How chromium-based software load and update data?



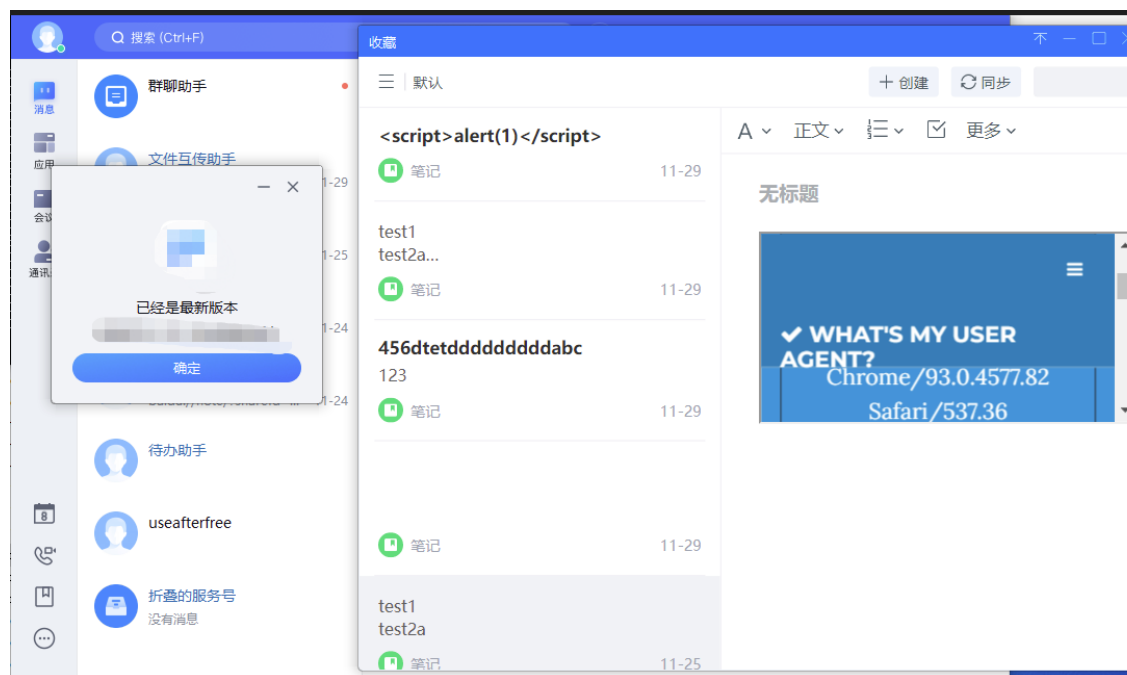
Scenario 3: Cache data injection

Cache data in chromium-base software

- How to find the cache file?
- What are the types of cache file?
 - Database
 - Json
 - custom format
 -



id	notelid	title	abstract	content	contentMD5	isSticked	isShared	isSyncContent	isLocalEdit	state	format	createTime	update
1	694204266902196224	<script>alert(1)</script>		...	d41d8cd98f00b204e9800998ecf8427e	0	1	0	0	2	1	1669696400474	1669711
2	694162351490076672			<p>img real-id="rid-0" class="attachment" ...	eaf8a669370ab65139645cd96497ef3cd	0	1	0	0	2	1	1669696400474	1669701
3	692693741571678208	456dtetdddddddddabc	123	<body>...	75b14934b0a18d072f4b9de331d5dced	0	1	0	0	2	1	1669346254740	1669701
4	692421780484399104	11	test1...	<body>...	fc6af6a6b6ae48ef87c7de48ece5a4a4	0	0	0	0	2	1	1669281416303	166934
5	686907635592859648		test1...	<body>...	5625dcb7cae6b646b0c42a5ec1101414	0	0	0	0	2	1	1667966741479	1669701
6	692697064693571584		test1...	<iframe src=http://whatsmyuseragent.org-></iframe>	cd7a3e844461e61e0f7501f59983107	0	0	0	0	2	1	1669347049218	166934



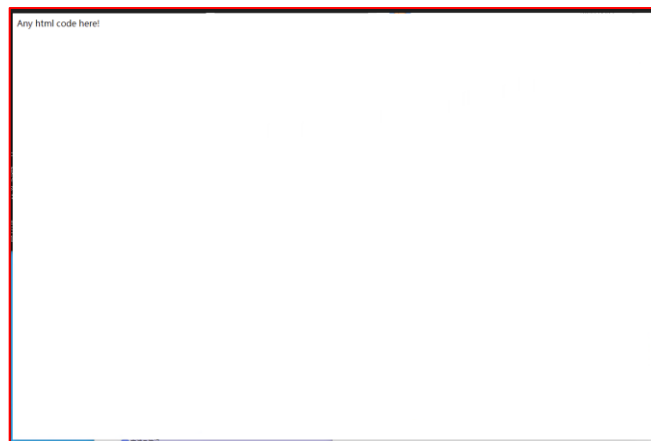
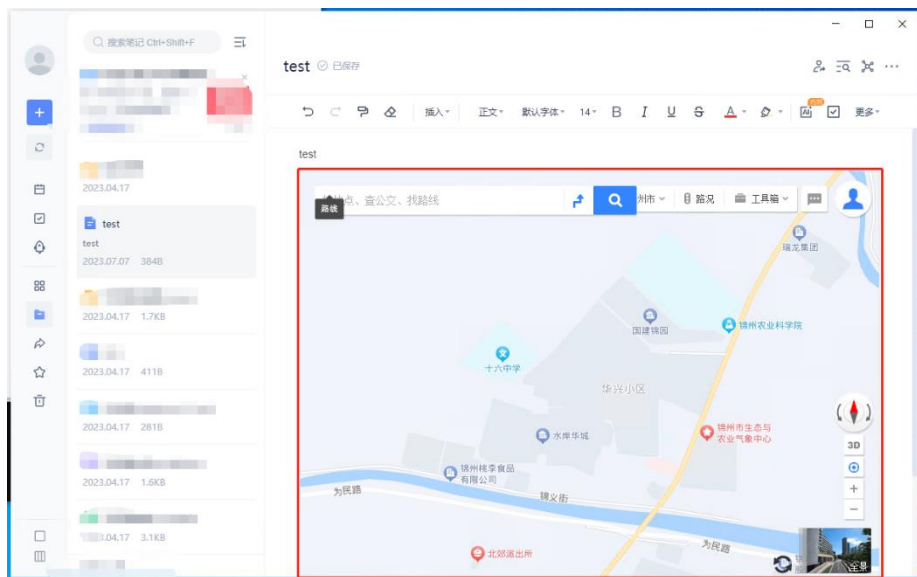
Scenario 3: Cache data injection

An interesting case

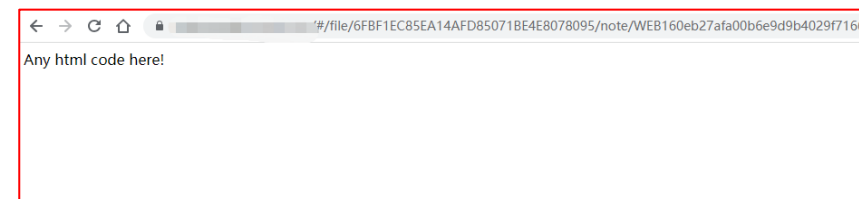
- From Local cache modify to server store XSS

```
WEB160eb27afa00b6e9d9b4029f7166b095 WEBc4085f0d7a88475c8fefa5e772c6fd2  
{ "2": "1", "3": "CWIf-1686033532478", "4": { "version": 1, "incompatibleVersion": 0, "fv": "0", "5": { { "3": "QFRF-1686033532480", "4": { "1": "h1", "5": { { "2": "2", "3":  
: "ub0x-1686033532479", "7": { { "8": "12131" } } }, "6": "h", { "3": "HsFW-1688962072020", "5": { { "2": "2", "3": "XhAu-1688962072021" } } }, { "3": "UQ4i-1688962031330", "  
4": { "1": "h1", "5": { { "2": "2", "3": "aco4-1688962031327" } } }, "6": "h", { "3": "meod-1686033532481", "4": { "u": "123", "6": "im", { "3": "IPW7-1686033624492", "5": { {  
"2": "2", "3": "4w8o-1686033624493" } } }, { "3": "gB0m-1686033648368", "4": { "mode": "Embed", "hf": "https://map. com/", "sr": "baidu-map", "6": "media", { "3":  
"iJQk-1686033648404", "5": { { "2": "2", "3": "WgpT-1686033648401" } } }, "title": "", "__compress__": "true" }
```

javascript:window.parent.parent.document.write('Any html code here!')



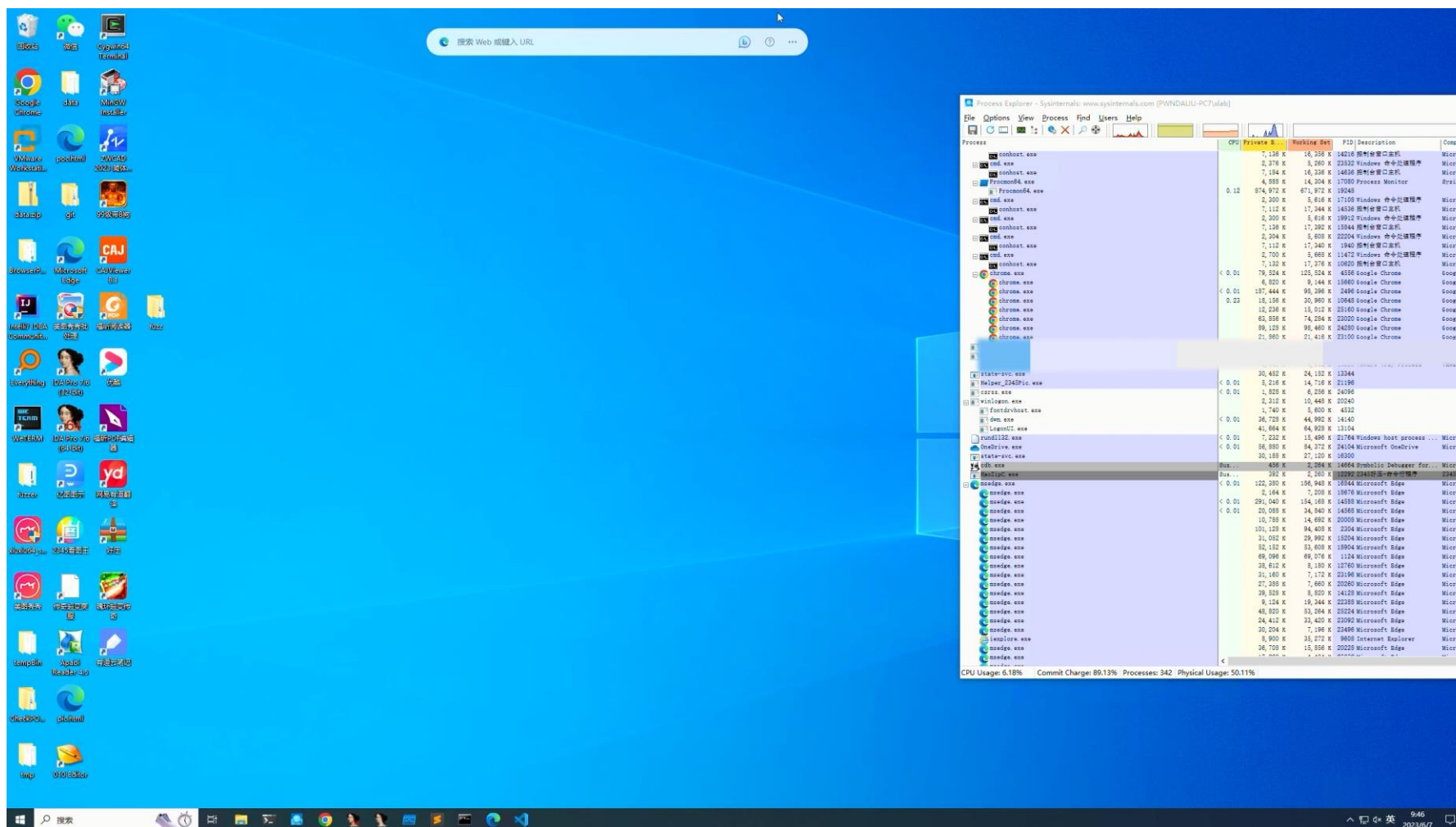
```
<div>  
<div class="media-print-placeholder css-liocsto">javascript:window.parent.parent.document.write('Any html code here!')</div> == $0  
<div class="css-aec4n3">
```



Scenario 3: Cache data injection

An interesting case

- From Local cache modify to RCE



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Scenario 4: plugin store and other open platform

Plugin store and other open platform

- Plugins give developers more functionality
- Plugins usually need to be officially reviewed before put on the store
- Has the convenience of a watering hole attack
- Hackers ***may*** use browser components to bypass plugin rereviews



Scenario 4: plugin store and other open platform

Plugin store and other open platform (demo)

The screenshot displays a Windows 10 desktop environment. In the foreground, a Windows Task Manager window is open, showing a list of running processes. The 'Processes' tab is selected, and the list is sorted by name. The processes include various system and user applications, such as 'svchost.exe', 'explorer.exe', 'chrome.exe', and 'cmd.exe'. The task manager window is partially obscured by a command prompt window in the background. The command prompt window is titled 'C:\Windows\System32\cmd.exe' and shows the command 'python -m http.server 8999' being executed. The desktop background is the standard Windows 10 blue wallpaper with the Start button and taskbar visible at the bottom. The taskbar shows the search bar, task view button, and several pinned applications including Edge, File Explorer, and the Task Manager.

Process Name	Private Bytes	Working Set	Session ID	Company Name	Product Name	Version	Architecture	Platform
svchost.exe	1,792 K	3,488 K	8204	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	16,024 K	44,608 K	8392	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	4,532 K	17,380 K	8352	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	5,076 K	12,444 K	8244	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	4,208 K	23,092 K	8304	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
SearchIndexer.exe	40,644 K	77,092 K	8376	Microsoft Corporation	Windows 搜索索引器	6.0.6002.18000	x64	Medium
SearchProtocolHost.exe	2,656 K	14,088 K	14448	Microsoft Corporation	Windows 搜索协议主机	6.0.6002.18000	x64	Medium
svchost.exe	2,616 K	10,588 K	14816	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	5,940 K	24,350 K	9408	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
SecurityHealthService.exe	7,140 K	20,200 K	3024	Microsoft Corporation	Windows 安全健康服务	6.0.6002.18000	x64	High
svchost.exe	4,080 K	20,824 K	10004	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	2,096 K	10,712 K	10020	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	1,600 K	11,288 K	8720	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	High
svchost.exe	6,344 K	23,212 K	2684	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,372 K	12,020 K	12408	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	5,484 K	20,272 K	12740	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	6,280 K	19,236 K	3540	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	3,020 K	12,488 K	3076	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	5,740 K	8,424 K	4180	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	3,196 K	11,408 K	11020	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	6,976 K	29,896 K	12568	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	42,472 K	83,120 K	7288	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	17,516 K	41,148 K	4716	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	7,064 K	21,192 K	8928	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,940 K	8,040 K	8140	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,044 K	10,144 K	3964	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	1,460 K	6,200 K	11788	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	41,476 K	44,840 K	11616	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,236 K	10,200 K	11396	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	1,792 K	7,420 K	14620	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	5,160 K	20,184 K	550	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	1,660 K	4,536 K	816	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	1,700 K	5,124 K	844	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,392 K	11,904 K	1080	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	1,720 K	4,648 K	1132	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	38,896 K	67,248 K	1280	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	32,800 K	50,176 K	1288	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,584 K	6,582 K	4928	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,644 K	11,124 K	8976	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	4,100 K	12,922 K	7140	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	141,024 K	91,140 K	3656	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	234,772 K	224,000 K	8504	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	1,948 K	9,944 K	2808	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,840 K	13,176 K	3320	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	4,344 K	14,068 K	11872	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,848 K	12,476 K	12004	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	33,300 K	97,156 K	3336	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	31,848 K	55,164 K	12340	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,180 K	4,688 K	5880	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	9,892 K	19,124 K	10064	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,816 K	11,892 K	12372	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium
svchost.exe	2,116,444 K	40,894 K	6540	Microsoft Corporation	Windows 服务主机进程	6.0.6002.18000	x64	Medium

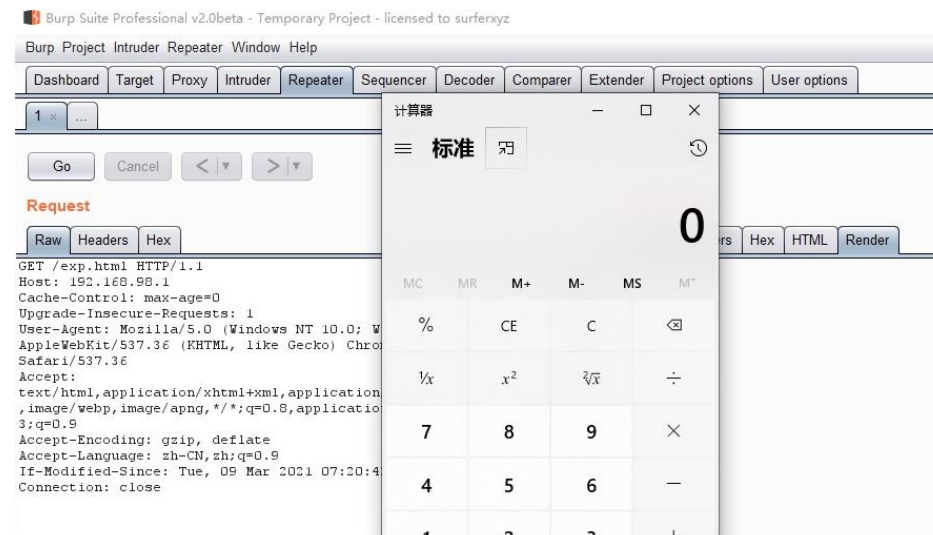
Scenario 5: other software feature

other software feature

Some functions may have an arbitrary page loading process, such as pre-rendering of crawlers/notes.

- Burp Suite RCE in 2021

-



A general V8 UAF exploit method in 32 bit

As the RCE window period is short, we need to write exp ASAP.

Typing bug => foo the Turbofan to make inconsistent scenes with JIT function, resulting
Type Confusion

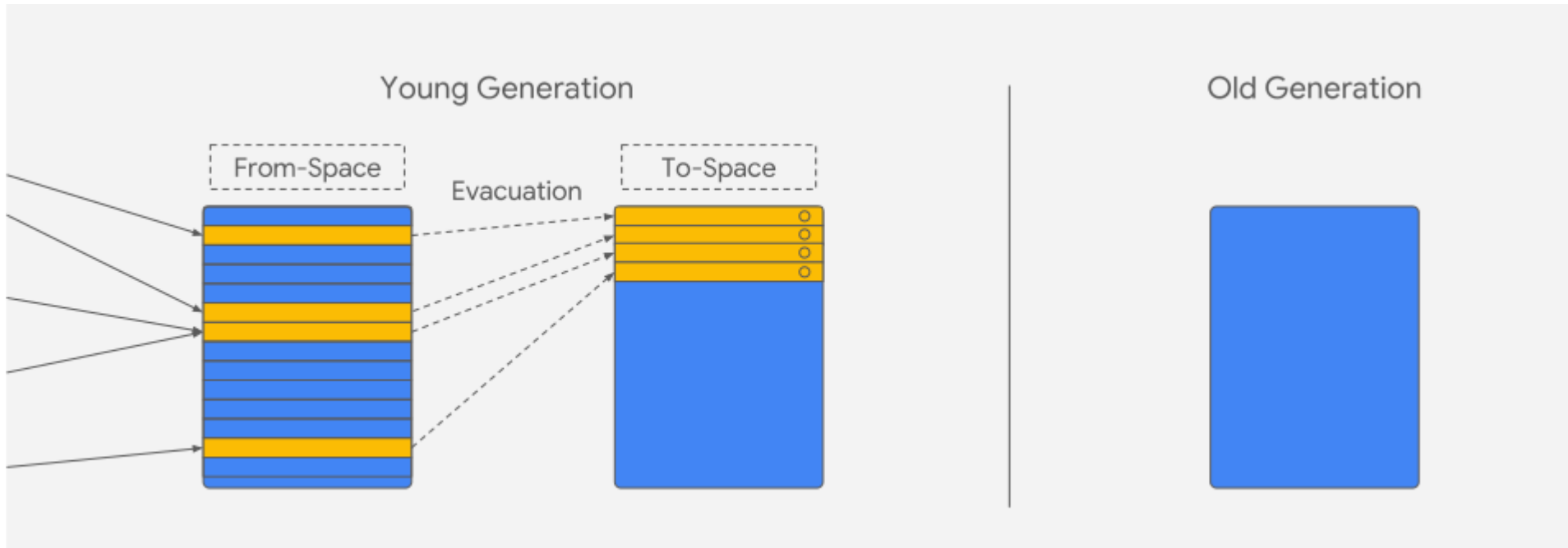
What about Use-after-Free? 🤔



A general V8 UAF exploit method in 32 bit

CVE-2022-1310(issue-1307610)

Overview about V8 Minor GC : track references from old-space-object to new-space-object by write-barrier



A general V8 UAF exploit method in 32 bit

CVE-2022-1310(issue-1307610)

Root cause : lack of write-barrier in RegExpUtils::SetLastIndex

```
MaybeHandle<Object> RegExpUtils::SetLastIndex(Isolate* isolate,
                                                Handle<JSReceiver> recv,
                                                uint64_t value) {
    Handle<Object> value_as_object =
        isolate->factory()->NewNumberFromInt64(value); /*** A ***/
    if (HasInitialRegExpMap(isolate, *recv)) { /*** B ***/
        JSRegExp::cast(*recv).set_last_index(*value_as_object,
        SKIP_WRITE_BARRIER); /*** C ***/
        return recv;
    } else {
        return Object::SetProperty(
            isolate, recv, isolate->factory()->lastIndex_string(),
            value_as_object,
            StoreOrigin::kMaybeKeyed, Just(kThrowOnError));
    }
}
```



A general V8 UAF exploit method in 32 bit

V8 ONE_BYTE_INTERNALIZED_STRING_TYPE
String layout : map | hash | length | content

```
gdb-peda$ telescope 0x5bc922bd-1
0000| 0x5bc922bc --> 0x370c21c9 -->
0004| 0x5bc922c0 --> 0xc40d3bc2
0008| 0x5bc922c4 --> 0x6
0012| 0x5bc922c8 ("cook1e")
0016| 0x5bc922cc --> 0x6531 ('1e')
```

```
DebugPrint: 0x5bc922bd: [String] in OldSpace: #cook1e
0x370c21c9: [Map] in ReadOnlySpace
- type: ONE_BYTE_INTERNALIZED_STRING_TYPE
- instance size: variable
- elements kind: HOLEY_ELEMENTS
- unused property fields: 0
- enum length: invalid
- stable_map
- back pointer: 0x370c232d <undefined>
- prototype_validity cell: 0
- instance descriptors (own) #0: 0x370c2139 <Other heap object (STRONG_DESCRIPTOR_ARRAY_TYPE)>
- prototype: 0x370c21ad <null>
- constructor: 0x370c21ad <null>
- dependent code: 0x370c2131 <Other heap object (WEAK_FIXED_ARRAY_TYPE)>
- construction counter: 0
```



A general V8 UAF exploit method in 32 bit

Overview : fake one_byte_internalized_string

AddressOf:

1. Obj : {mark:0x13371337,prop:target_obj}
2. Fake string : find target object address by mark. 😊

FakeStr

StringMap

...

0x100000

AAAA

...

Obj

Obj Map

mark

Target obj Address

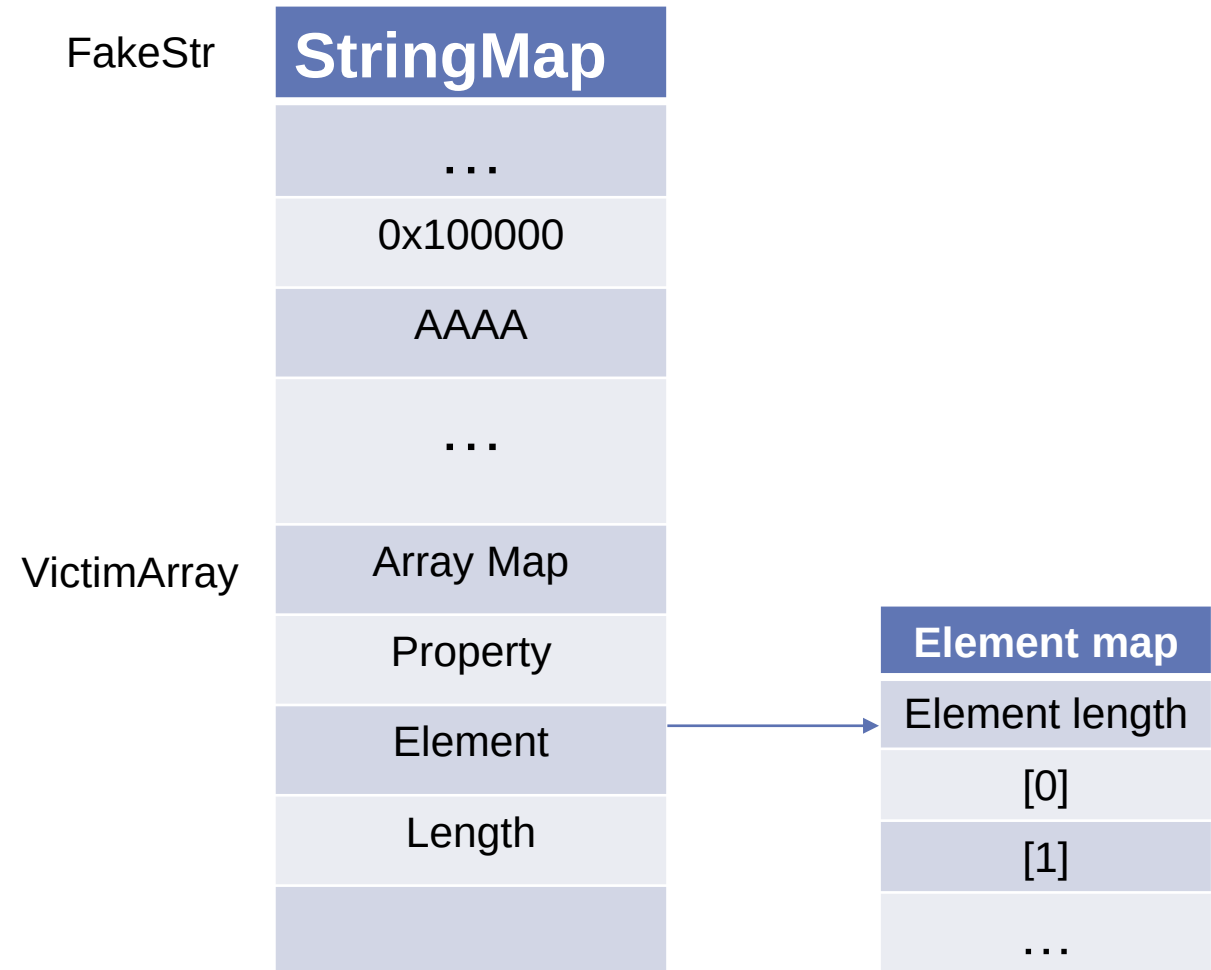


A general V8 UAF exploit method in 32 bit

Overview : fake one_byte_internalized_string

ArbitraryRead/ArbitraryWrite:

1. Hijack VictimArray->Element to target_addr-0x10
2. Read/write VictimArray[0] will directly read/write target_addr



A general V8 UAF exploit method in 32 bit

But how to fake string?

V8 object use map to identify the type, so the key is to fake string map.

HeapSpray:

1. Stable address : Spray Heap with ArrayBuffer and we can get a stable address with debugger
2. Fake string map in this memory : fake string map's prototype、constructor and the map of map.
3. Spray FakeStr to occupy the RegExp.lastIndex with JsArray

```
0:026> dds 0x33c824cd-1 L8
33c824cc 20cf4001
33c824d0 20cf4001
33c824d4 00030000
33c824d8 20cf4001
33c824dc 20cf4001
33c824e0 20cf4001
33c824e4 00030000
33c824e8 20cf4001
```

```
0x33c824cd <String[196608]: #...<truncated>>>
```



A general V8 UAF exploit method in 32 bit

RCE on 91.0.4472.0(demo)

```
C:\Windows\System32\cmd.exe
D:\browser\Win_870768_chrome-win\chrome-win>

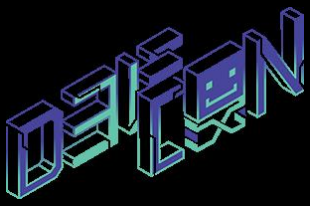
C:\Windows\System32\cmd.exe - python3 -m http.server 1337 - python3 -m http.server 1337
D:\speech\defcon-submission\UAF_32bit>python3 -m http.server 1337
Serving HTTP on :: port 1337 (http://[::]:1337/) ...
::ffff:127.0.0.1 - - [25/Jul/2023 16:54:40] "GET /index.html HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [25/Jul/2023 16:55:26] code 404, message File not found
::ffff:127.0.0.1 - - [25/Jul/2023 16:55:26] "GET /favicon.ico HTTP/1.1" 404 -
```



Summary

- A little bug or feature can lead to RCE.
- Lack of attention to the attack path of browser vulnerabilities
- The attack surface is relatively rudimentary
- Hackers also can bypass mitigation measures directly using powerful Chromium bugs.





3. Achieve RCE Without Sandbox Escape Bug

The sandbox in chromium

- Mandatory access controlled environment
- Isolated Process when HTML rendering and JavaScript execution
- Limited resource access & Limited IPC/kernel interaction access

OS	Sandbox implementation
Windows	A restricted token& The Windows job object & The Windows desktop object& Integrity levels&
Linux	Seccomp-BPF & User namespaces
Android	SELinux



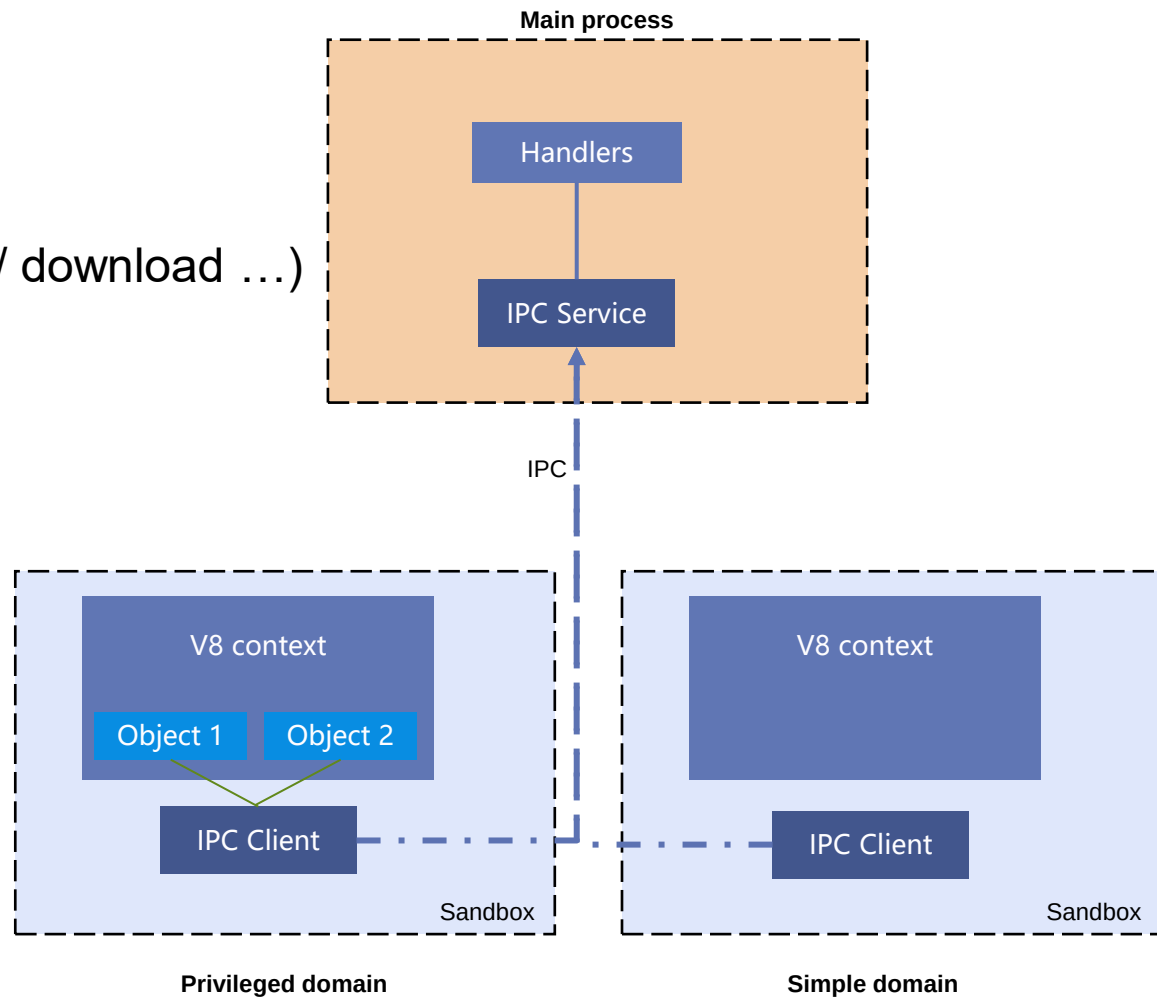
How can we achieve RCE in sandbox-enabled software without a chromium sandbox-escape bug?

What's new in chromium-based software?

Privileged API

- client-side behaviors in specific scenarios(upload / download ...)
- For renderer, it is a series of JavaScript objects
- For Browser, it is a series of handler code

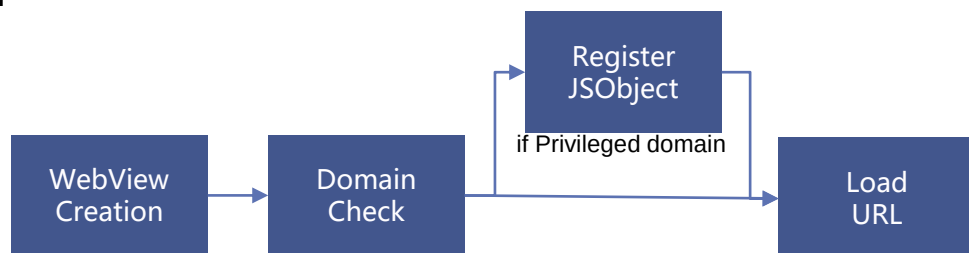
Type	Privileged API implementation
Electron	preload.js
CEF	CefV8Handler
Android	JavaScriptInterface



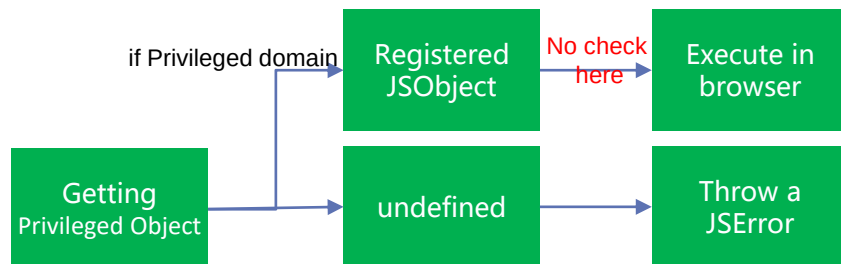
What's new in chromium-based software?

The development logic

1.



Initialization

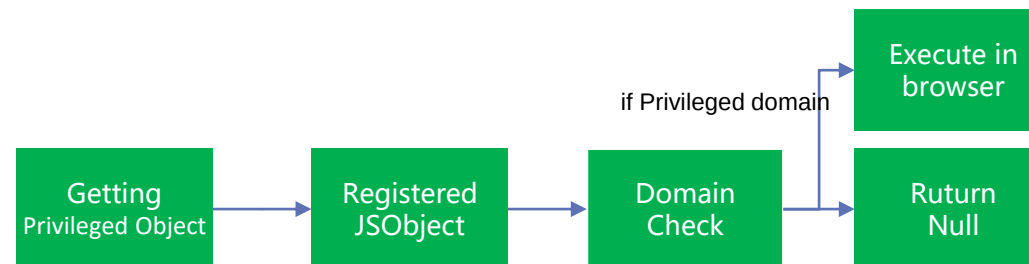


Runtime

2.



Initialization



Runtime

What if a non-privileged WebView get a privileged context?

How to properly determine if IPC access is from a privileged domain?



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Sandbox Escape In Electron

Sandbox Escape with the lack of context isolation

context isolation : separate JS context between page's scripts and preload scripts, introduced in electron 1.4.15 and enabled by default since Electron 12

Preload.js : has access to:

- (1) Some Node.js native classes like **require**
- (2) Some Electron modules like **ipcRenderer**

Attacker can use Prototype Pollution to make any ipc call.

[13] : <https://speakerdeck.com/masatokinugawa/electron-abusing-the-lack-of-context-isolation-curecon-en?slide=2>

[14] : <https://i.blackhat.com/asia-19/Thu-March-28/bh-asia-Carettoni-Preloading-Insecurity-In-Your-Electron.pdf>



Sandbox Escape In Electron

CVE-2022-29247

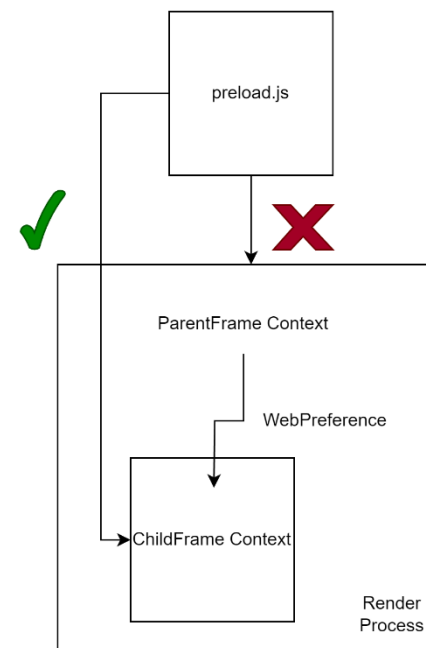
Overview : Disabling context isolation, node_integration_in_sub_frames using an unauthorised frame
In a comprised render process, attackers overwrite the context's prefs.node_integration_in_sub_frames to load preload.js in Child Frame

```
diff --git a/third_party/blink/common/web_preferences/web_preferences.cc b/third_party/blink/common/web_preferences/web_preferences.cc
index 1e51bb68df930c44e165b32c8c03b99986792209..9535330e3e7a3c4621e
--- a/third_party/blink/common/web_preferences/web_preferences.cc
+++ b/third_party/blink/common/web_preferences/web_preferences.cc
@@ -143,6 +143,29 @@ WebPreferences::WebPreferences()
     navigate_on_drag_drop(true),
     v8_cache_options(blink::mojom::V8CacheOptions::kDefault),
     record_whole_document(false),
+
+    // Begin Electron-specific WebPreferences.
+    disable_electron_site_instance_overrides(),
+    background_color(base::EmptyString()),
+    opener_id(0),
+    context_isolation(false),
+    enable_remote_module(false),
+    world_safe_execute_javascript(false),
+    guest_instance_id(0),
+    hidden_page(false),
+    offscreen(false),
+    preload(base::FilePath::StringType()),
+    native_window_open(false),
+    node_integration(false),
+    node_integration_in_worker(false),
+    node_leakage_in_renderers(false),
+    node_integration_in_sub_frames(false),
+    enable_spellcheck(false),
+    enable_plugins(false),
+    enable_websql(false),
+    webview_tag(false),
+    // End Electron-specific WebPreferences.
+}
```

```
void ElectronSandboxedRendererClient::DidCreateScriptContext(
    v8::Handle<v8::Context> context,
    content::RenderFrame* render_frame) {
    // Only allow preload for the main frame or
    // For devtools we still want to run the preload_bundle script
    // Or when nodeSupport is explicitly enabled in sub frames
    bool is_main_frame = render_frame->IsMainFrame();
    bool is_devtools =
        IsDevTools(render_frame) || IsDevToolsExtension(render_frame);

    bool allow_node_in_sub_frames =
        render_frame->GetBlinkPreferences().node_integration_in_sub_frames;

    bool should_load_preload =
        (is_main_frame || is_devtools || allow_node_in_sub_frames) &&
        !IsWebViewFrame(context, render_frame);
    if (!should_load_preload)
        return;
```



Sandbox Escape In Electron

CVE-2022-29247

Overview : Disabling context isolation, node_integration_in_sub_frames using an unauthorised frame
In a comprised render, it overwrite the context's prefs.context_isolation and disable CISO in new renderer process.

Then we can send unauthorized IPC call with prototype pollution.

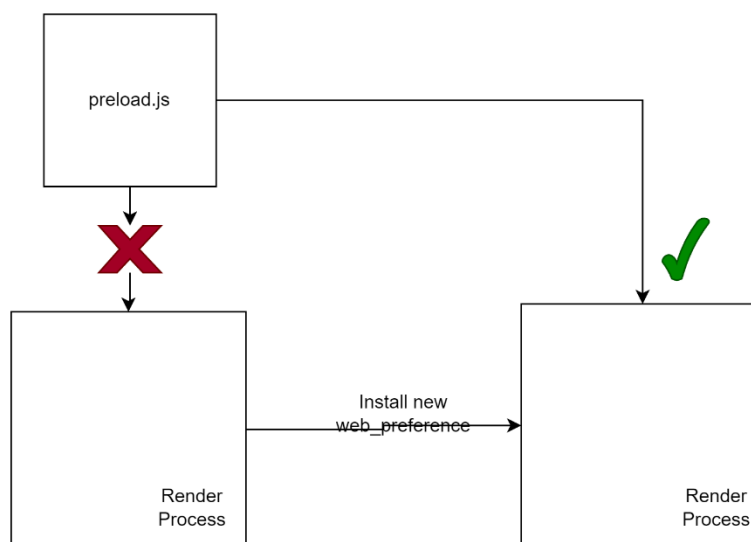
```
void ElectronRenderFrameObserver::DidInstallConditionalFeatures(
    v8::Handle<v8::Context> context,
    int world_id) {
    auto* isolate = context->GetIsolate();
    v8::MicrotasksScope microtasks_scope(
        isolate, v8::MicrotasksScope::kDoNotRunMicrotasks);

    if (ShouldNotifyClient(world_id))
        renderer_client->DidCreateScriptContext(context, render_frame_);

    auto prefs = render_frame->GetBlinkPreferences();
    [0] bool use_context_isolation = prefs.context_isolation;
    // This logic matches the EXPLAINED logic in electron_renderer_client.cc
    // to avoid explaining it twice go check that implementation in
    // DidCreateScriptContext();
    bool is_main_world = IsMainWorld(world_id);
    bool is_main_frame = render_frame->IsMainFrame();
    bool allow_node_in_sub_frames = prefs.node_integration_in_sub_frames;

    bool should_create_isolated_context =
        use_context_isolation && is_main_world &&
        (is_main_frame || allow_node_in_sub_frames);

    if (should_create_isolated_context) {
        [1] CreateIsolatedWorldContext();
        if (!renderer_client->IsWebViewFrame(context, render_frame_))
            renderer_client->SetupMainWorldOverrides(context, render_frame_);
    }
}
```



Others





4. Security Recommendations

Security Recommendations

How to defend against “RCE window period”

- For programs that use Chromium components, it is recommended to promptly incorporate the latest version of the Chromium library.
- For software with high coupling, it may not be possible to update to a major version of Chromium. In this case, it is recommended to use a stable development version and introduce security patches with software updates.
- Enable the Chromium sandbox.
- Disable unused software functionalities or enable internal integrated security defense mechanisms.



Security Recommendations

Security Development Suggestions

- Add backend validation and frontend rendering validation.
- Follow the principle of least privilege, strictly control the list of privileged domains, and limit privileged API functionalities.
- Enable universal protection mechanisms, such as Content Security Policy (CSP).
- Be vigilant about input from unexpected sources.





5. Conclusion & Takeaways

Conclusion & Takeaways

In this topic:

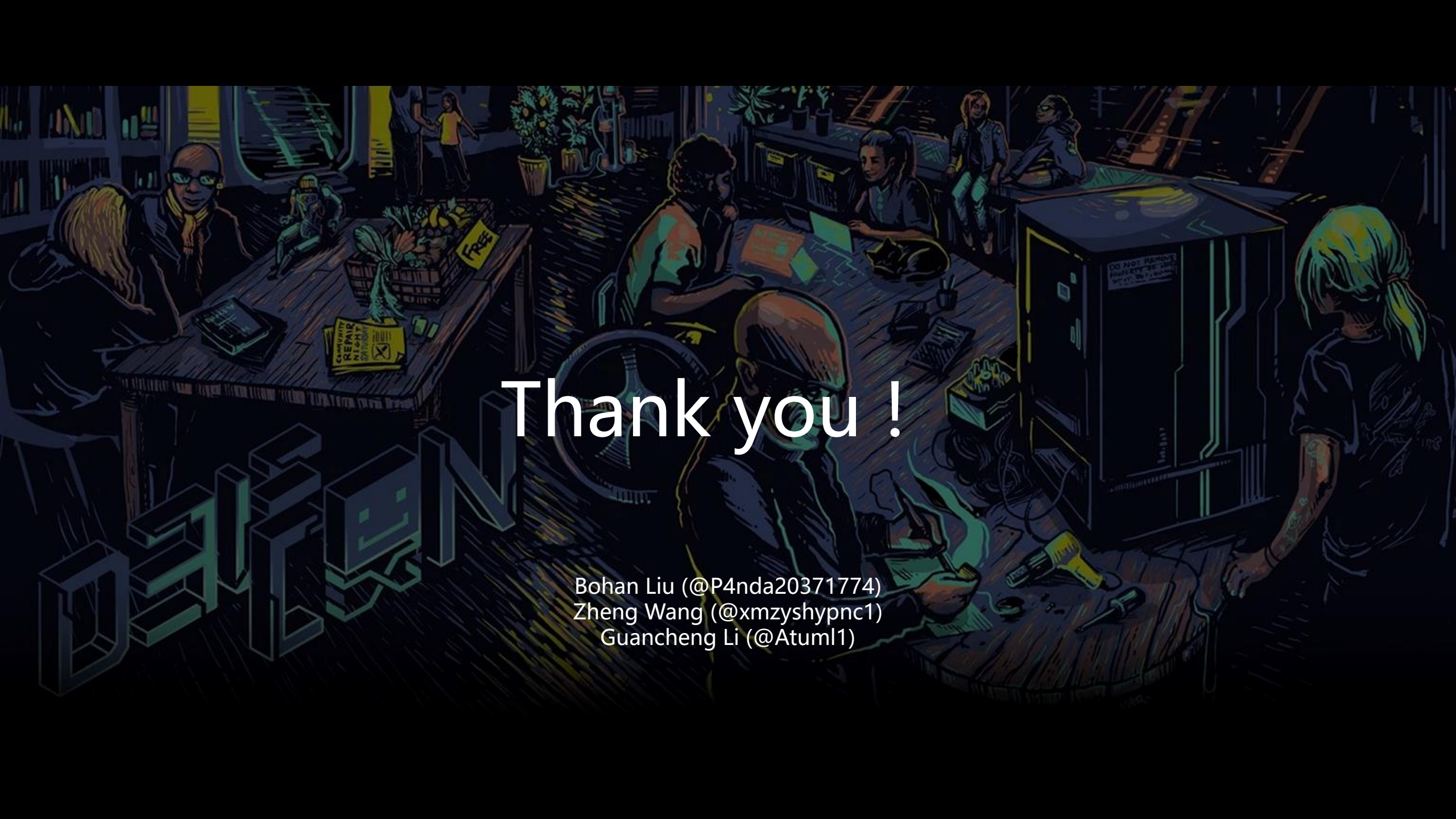
- Verify that a large number of chromium-based software have RCE window period.
- Summarize some software attack surfaces that cooperate with the RCE window period, and give real-world examples.
- Share a general V8 UAF exploit method in 32 bit.
- Provide some security recommendations to help strengthen the security of chromium-based software.



Acknowledgement

- Yang Yu (@tombkeeper)
- Kai Song (@ExpSky)
- Wei Liu
- Guannan Wang (@Keenan7310)
- ZeKai Wu (@hellowuzekai)
- Junyu Zhou(@md5_salt)
- Haibin Shi (@Aryb1n)
- Yongke Wang (@Rudykewang)
- Huiming Liu (@liuhm09)





Thank you !

Bohan Liu (@P4nda20371774)
Zheng Wang (@xmzyshypnc1)
Guancheng Li (@Atuml1)

References

- [1] https://www.cvedetails.com/product/15031/Google-Chrome.html?vendor_id=1224
- [2] <https://googleprojectzero.blogspot.com/p/0day.html>
- [3] <https://bughunters.google.com/about/rules/5745167867576320/chrome-vulnerability-reward-program-rules>
- [4] <https://blog.electrovolt.io/posts/discord-rce/>
- [5] <https://github.com/r4j0x00/exploits/commit/7ba55e5ab034d05877498e83f144e187d3ddb160>
- [6] <https://i.blackhat.com/USA-22/Thursd...sktop-Apps.pdf>
- [7] <https://github.com/pd4d10/debugtron>
- [8] <https://github.com/electron/asar>
- [9] <https://github.com/typora/typora-issues/issues/2959>
- [10] <https://bugs.chromium.org/p/chromium/issues/detail?id=722756>
- [11] <https://blog.noah.360.net/burp-suite-rce/>
- [12] <https://blackhat.com/asia-19/briefings/schedule/index.html>
- [13] <https://speakerdeck.com/masatokinugawa/electron-abusing-the-lack-of-context-isolation-curecon-en?slide=2>
- [14] <https://i.blackhat.com/asia-19/Thu-March-28/bh-asia-Carettoni-Preloading-Insecurity-In-Your-Electron.pdf>
- [15] <https://hackerone.com/s1r1u5>
- [16] <https://crbug.com/1307610>

